

第五章 运算方法与运算器(一)

秦磊华 计算机学院

基于补码数据表示研究运算方法和设计运算器(简)

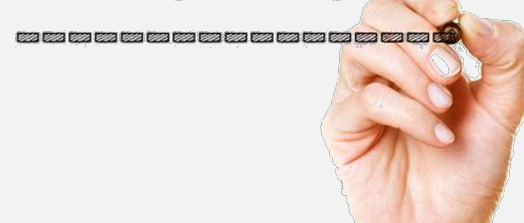
5.1 定点加法运算

5.2 加法运算器设计

5.3 定点减法运算

5.4 定点减法运算器设计

CONTENT



5.1 定点数加/减运算

1. 补码加法运算方法

$$[X + Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}}$$

1) 理解意义

2) 公式证明思路及可用的方法



$$[X]_{\text{补}} = \begin{cases} X & 0 \leq X < 2^n \\ 2^{n+1} + X & -2^n \leq X < 0 \end{cases} \quad \text{MOD } 2 \quad (2^{n+1})$$

$$[X]_{\text{补}} = \begin{cases} X & 0 \leq X < 1 \\ 2 + X & -1 \leq X < 0 \end{cases}$$

(Note: In the second formula, the 'X' in the first case is circled in red, and the '2' in the second case is also circled in red, indicating the modulo 2 operation.)



5.1 定点数加/减运算

$$[X + Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}}$$

- 1) $X > 0, Y > 0$ (可直接理解)
- 2) $X > 0, Y < 0$
- 3) $X < 0, Y > 0$ (2/3证明相同)
- 4) $X < 0, Y < 0$



5.1 定点数加/减运算

1) $X > 0, Y > 0$

因为: $x + y > 0$

所以: $x + y = [x + y]_{\text{补}} \pmod{2^{n+1}}$



5.1 定点数加/减运算

2) $X > 0, Y < 0$

所以: $[x]_{\text{补}} = x$ $[y]_{\text{补}} = 2^{n+1} + y$

$$\begin{aligned} [x]_{\text{补}} + [y]_{\text{补}} &= x + 2^{n+1} + y \\ &= 2^{n+1} + (x + y) \end{aligned}$$

当 $x + y < 0$ 时

$$2^{n+1} + (x + y) = [x + y]_{\text{补}} \pmod{2^{n+1}}$$

当 $x + y > 0$ 时

$$2^{n+1} + (x + y) > 2^{n+1} \quad \text{模舍去}$$

$$\begin{aligned} [x]_{\text{补}} + [y]_{\text{补}} &= 2^{n+1} + (x + y) = x + y \pmod{2^{n+1}} \\ &= [x + y]_{\text{补}} \pmod{2^{n+1}} \end{aligned}$$

3) $X < 0, Y > 0$

同左

4) $X < 0, Y < 0$

$$[x]_{\text{补}} = 2^{n+1} + x \quad [y]_{\text{补}} = 2^{n+1} + y$$

$$\begin{aligned} [x]_{\text{补}} + [y]_{\text{补}} &= (2^{n+1} + x) + (2^{n+1} + y) \\ &= 2^{n+1} + (2^{n+1} + x + y) \pmod{2^{n+1}} \end{aligned}$$

$$-2^{n+1} \leq x + y < 0$$

$$\text{故 } 0 \leq 2^{n+1} + x + y < 2^{n+1}$$

$$\begin{aligned} \text{故 } 2^{n+1} + (2^{n+1} + x + y) \pmod{2^{n+1}} &= (2^{n+1} + x + y) \\ &= [x + y]_{\text{补}} \pmod{2^{n+1}} \end{aligned}$$



5.1 定点数加/减运算

例1 已知 $X = -10111$, $Y = -1000$, 求 $X+Y$

解: $[X]_{\text{补}} = 101001$, $[Y]_{\text{补}} = 11000$

101001

+ 11000

011000

1100001

$X+Y = -11111$

2. 补码减法运算方法

$$[X-Y]_{补} = [X]_{补} - [Y]_{补} = [X]_{补} + [-Y]_{补}$$

1) 理解意义

2) 公式证明思路及可用的方法

$$\begin{aligned} [X-Y]_{补} &= [X+(-Y)]_{补} \\ &= [X]_{补} + [-Y]_{补} ! \end{aligned}$$

$$[Y-Y]_{补} = [Y]_{补} + [-Y]_{补} = 0$$

$$\hookrightarrow -[Y]_{补} = [-Y]_{补}$$





5.1 定点数加/减运算

2. 补码减法运算方法

$$[X-Y]_{\text{补}} = [X]_{\text{补}} - [Y]_{\text{补}} = [X]_{\text{补}} + [-Y]_{\text{补}}$$

例2 已知 $[X]_{\text{补}} = 101001$, 求 $[-X]_{\text{补}}$

由 $[X]_{\text{补}} = 101001$

└→ $X = -10111$

└→ $-X = 10111$

└→ $[-X]_{\text{补}} = 010111$

$\updownarrow$
 $[X]_{\text{补}} = 101001$

5.1 定点数加/减运算

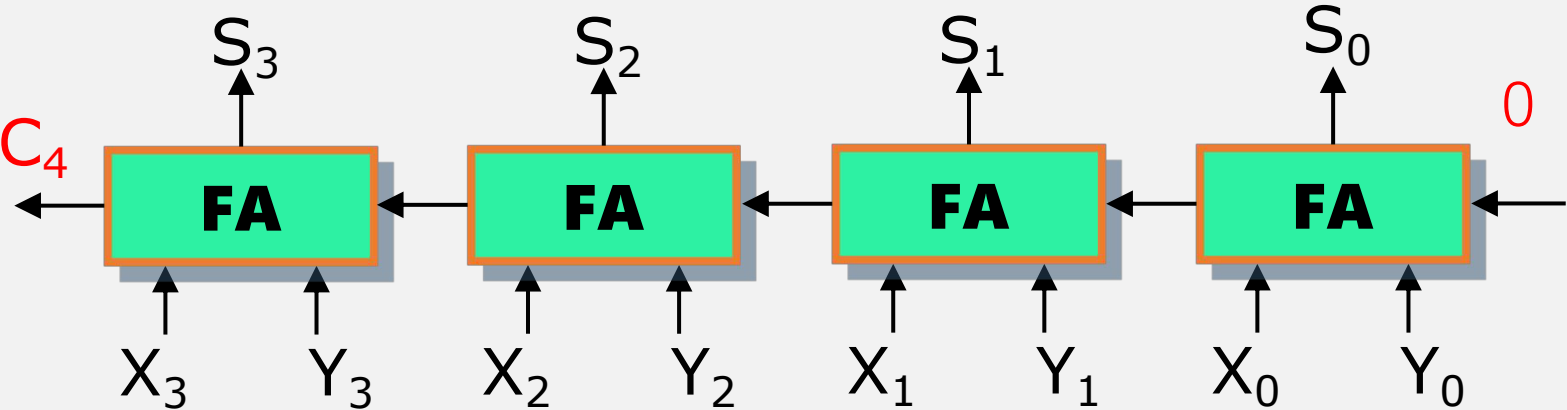
例3 已知 $X = -10111$, $Y = -1000$, 求 $X - Y$

解: $[X]_{\text{补}} = 101001$, $[Y]_{\text{补}} = 11000$ $[-Y]_{\text{补}} = 01000$

$$\begin{array}{r} 1\ 0\ 1\ 0\ 0\ 1 \\ +\ 0\ 0\ 1\ 0\ 0\ 0 \\ \hline 0\ 0\ 1\ 0\ 0\ 0 \\ \hline 1\ 1\ 0\ 0\ 0\ 1 \end{array}$$

$$X - Y = -1111$$

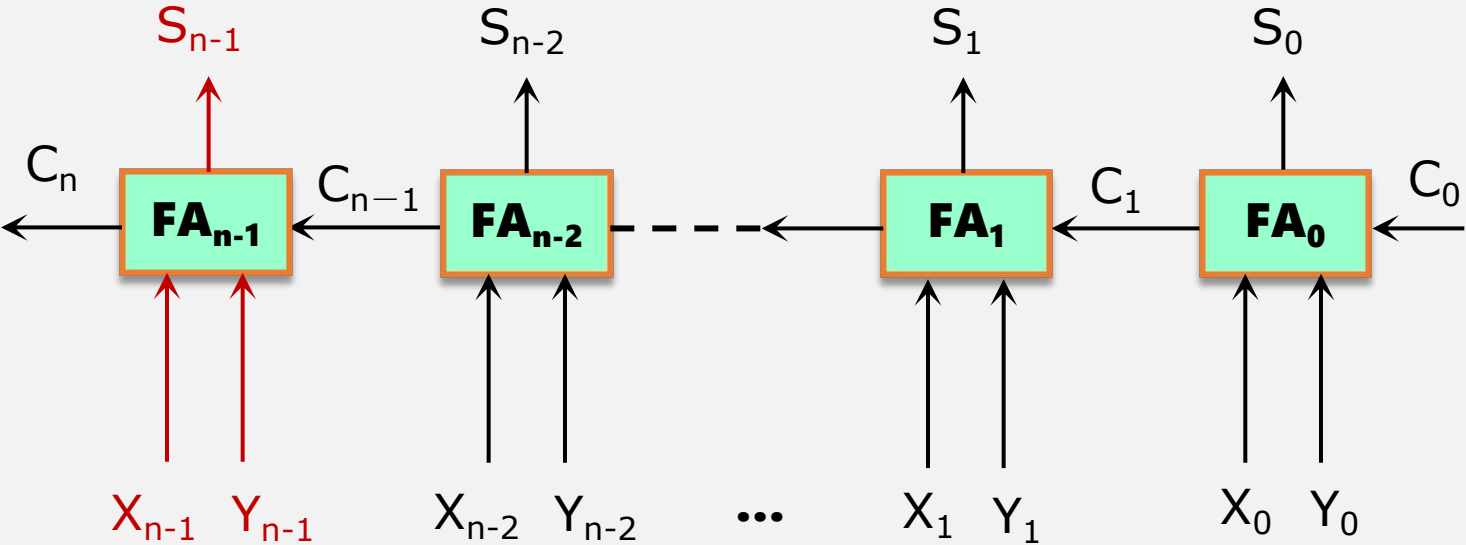
3. 基本补码加法运算器设计





5.1 定点数加/减运算

3. 基本补码加法运算器设计

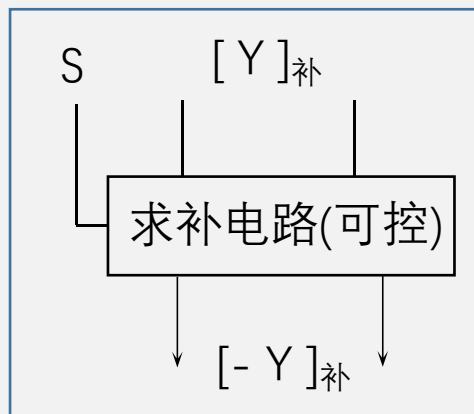
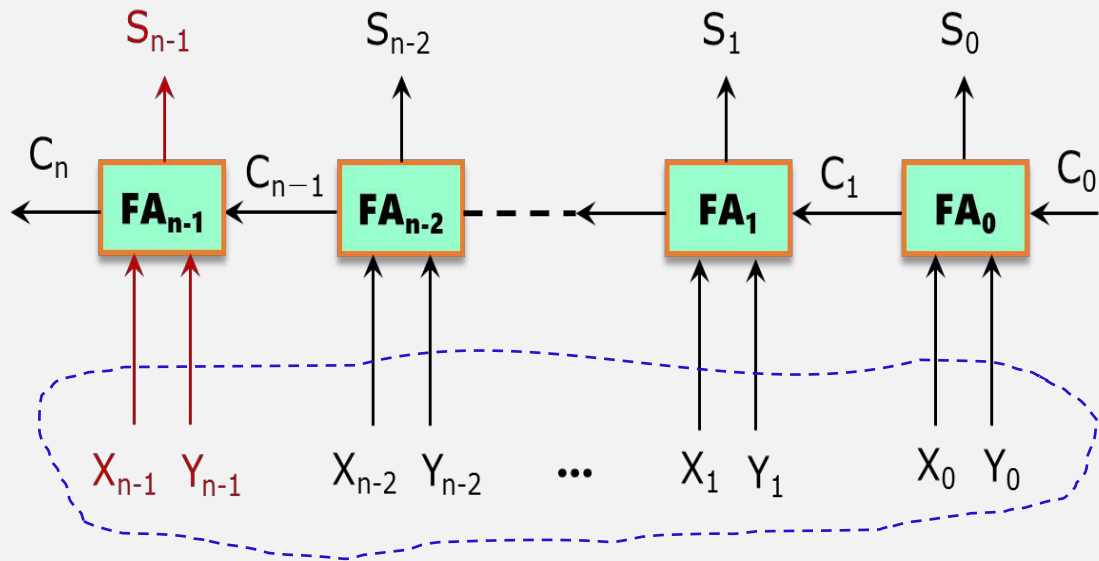


5.1 定点数加/减运算

3. 基本补码减法运算器设计

$$[X + Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}}$$

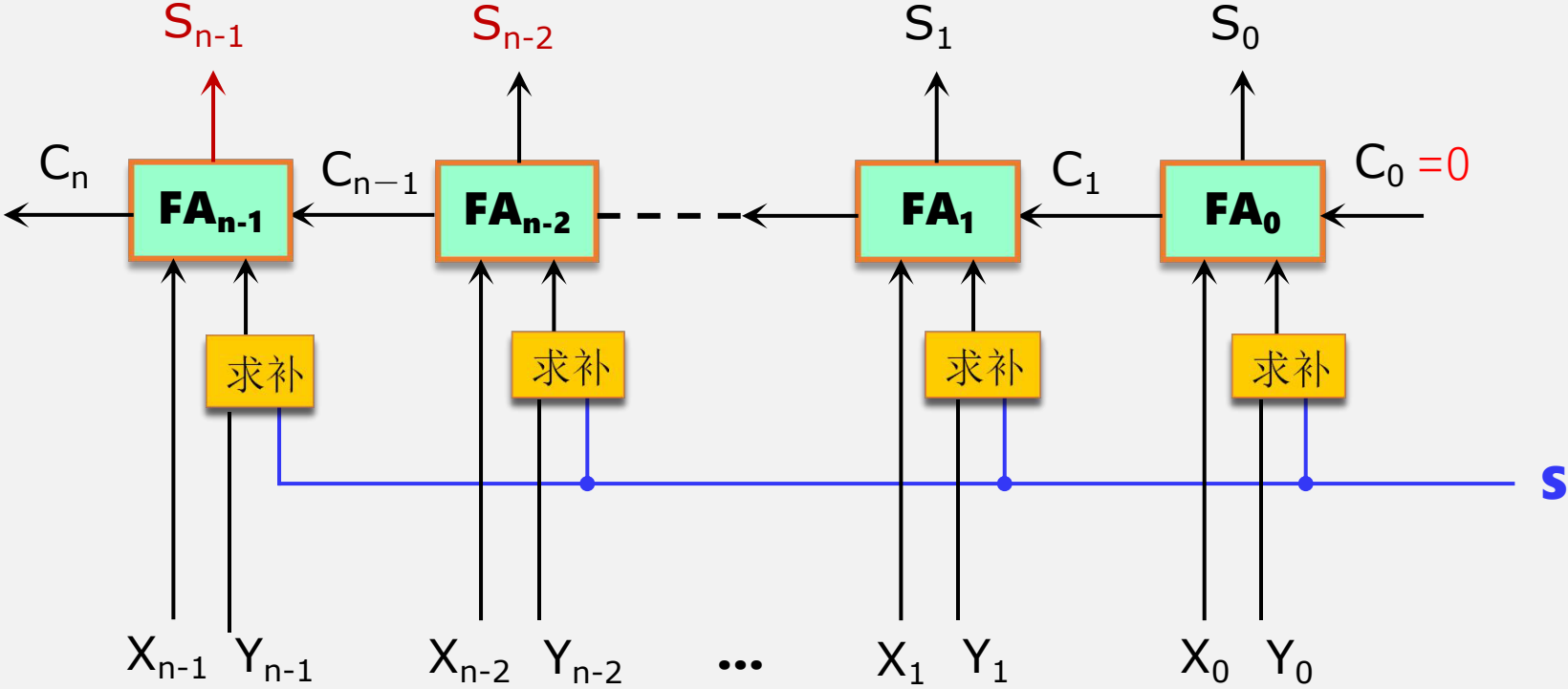
$$[X - Y]_{\text{补}} = [X]_{\text{补}} - [Y]_{\text{补}} = [X]_{\text{补}} + [-Y]_{\text{补}}$$





5.1 定点数加/减运算

3. 基本补码减法运算器设计

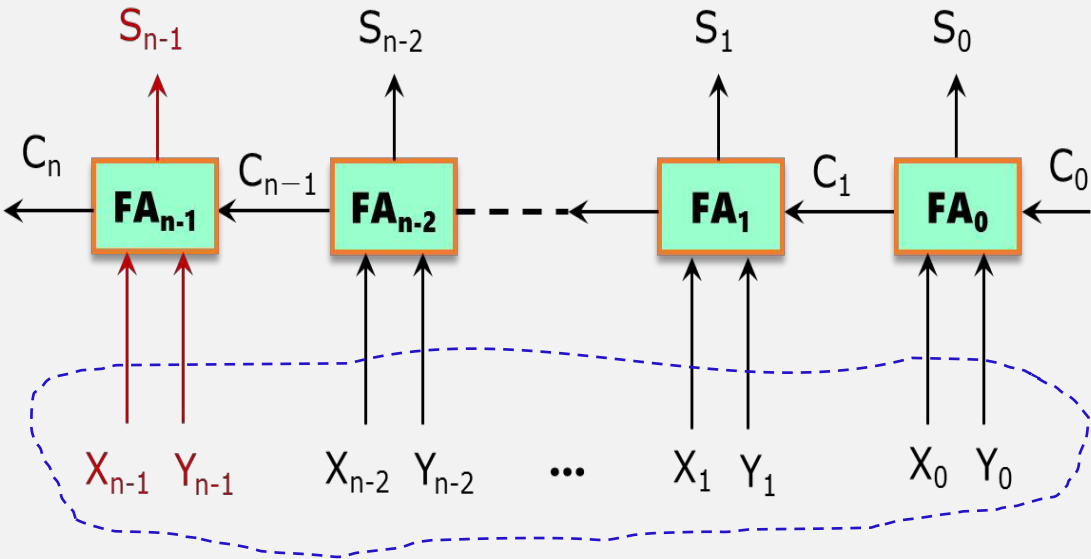


S = 0 : 加法
S = 1 : 减法
问题?

3. 基本补码减法运算器设计

$$[X + Y]_{补} = [X]_{补} + [Y]_{补}$$

$$[X - Y]_{补} = [X]_{补} - [Y]_{补} = [X]_{补} + [-Y]_{补}$$

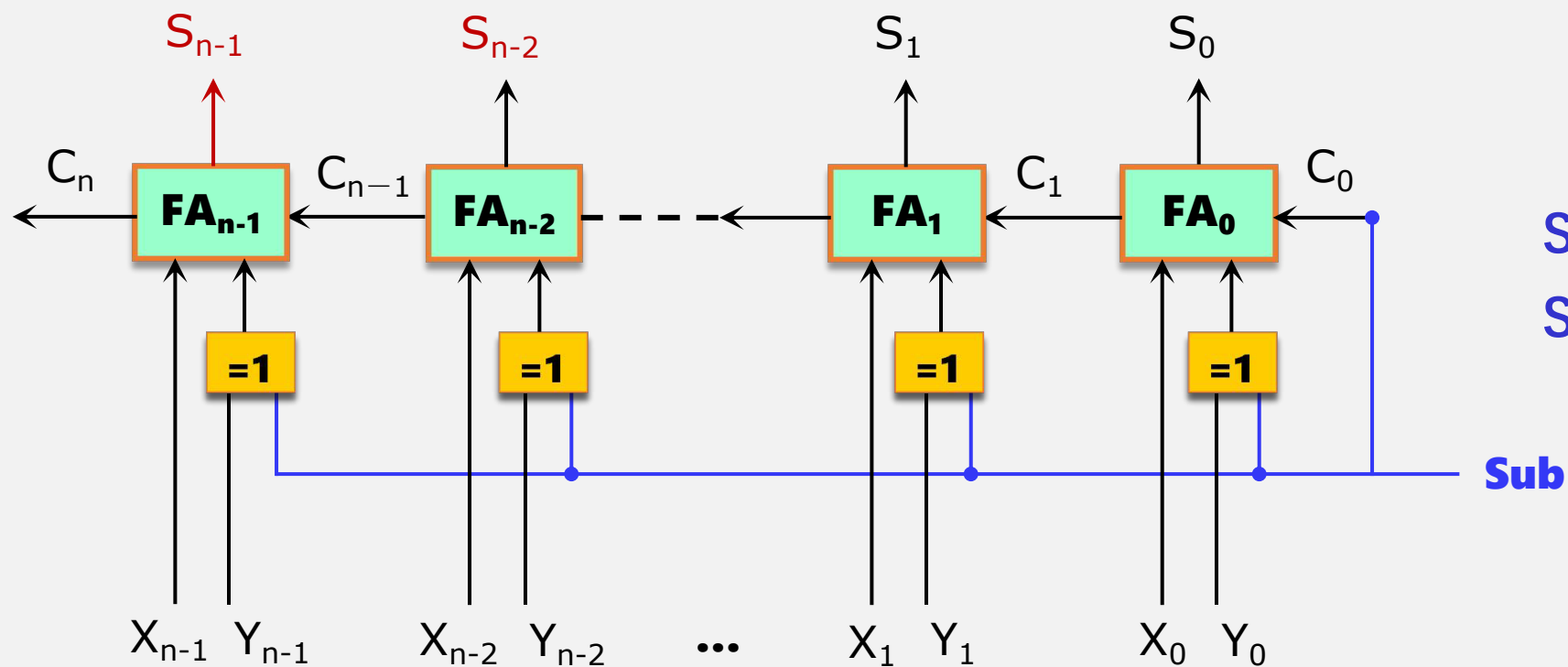


$$\begin{array}{r}
 1101101 \\
 \oplus 1111111 \\
 \hline
 0010010
 \end{array}$$

$$\begin{array}{r}
 1101101 \\
 \oplus 0000000 \\
 \hline
 1101101
 \end{array}$$

5.1 定点数加/减运算

3. 基本补码减法运算器设计



Sub = 0 : 加法
 Sub = 1 : 减法

减法不满足交换律



5.1 定点数加/减运算

4. 补码加减法运算溢出

例4 已知 $X = -11011$, $Y = -1000$ 求 $X + Y$

解: $[X]_{\text{补}} = 100101$, $[Y]_{\text{补}} = 11000$

100101

+ 111000

1

~~0~~

11101

$X+Y = +11101$



5.1 定点数加/减运算

4. 补码加减法运算溢出

例5 已知 $X = +10101$, $Y = +10000$ 求 $X + Y$

解: $[X]_{\text{补}} = 010101$, $[Y]_{\text{补}} = 01000$

0

1

0

1

0

1

+

0

1

1

0

0

0

1

0

1

1

0

1

$X+Y = -10011$



5.1 定点数加/减运算

4. 补码加减法运算溢出

	0	1	0	1	0	1
+	0	1	1	0	0	0
	1	0	1	1	0	1

	1	0	0	1	0	1
+	1	1	1	0	0	0
1	0	1	1	1	0	1

溢出: 运算结果超出对应数据类型和机器字长的数据表示范围

5. 补码加减法运算溢出检测

◆ 根据概念，溢出只发生在同号数相加时，包括 $[X]_{补}$ 与 $[Y]_{补}$ ， $[X]_{补}$ 与 $[-Y]$ 同号

1) 方法1：对操作数和运算结果的符号位进行检测

当结果的符号与操作数的符号不同时表明发生了溢出

	0	1	0	1	0	1
+	0	1	1	0	0	0
	1	0	1	1	0	1

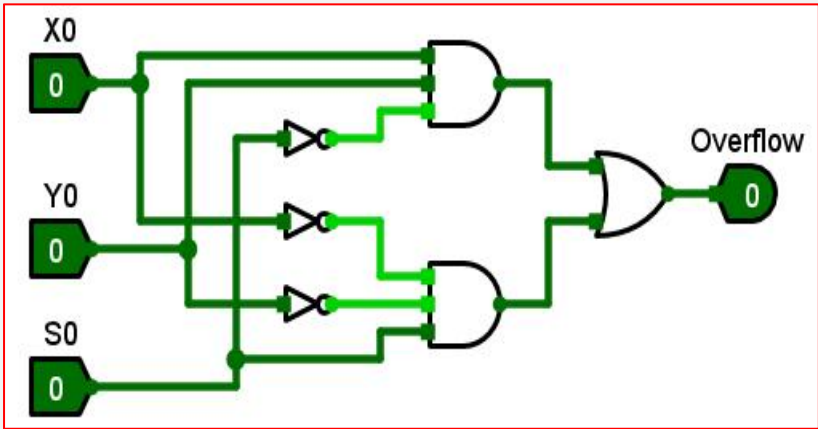
	1	0	0	1	0	1
+	1	1	1	0	0	0
	1	0	1	1	1	0

5. 补码加减法运算溢出检测

设 X_n ， Y_n 为参加运算数的符号位， S_n 为结果符号位

$$\text{Overflow} = X_n Y_n \overline{S_n} + \overline{X_n} \overline{Y_n} S_n$$

当Overflow=1时， 结果溢出， 根据逻辑表达式， 得到相应电路

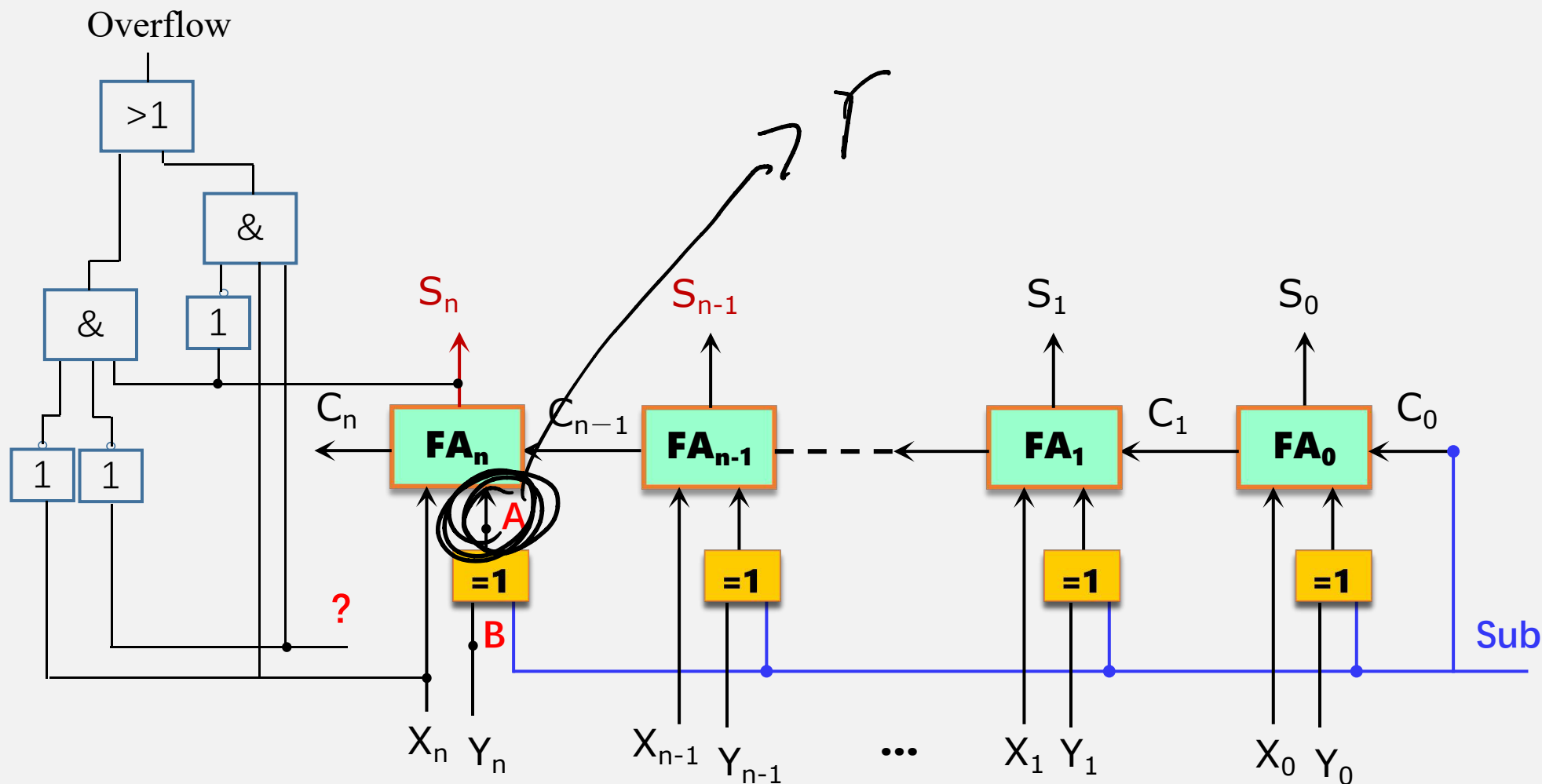


代价分析:
6个逻辑门， 3级时延

5.1 定点数加/减运算

5. 补码加减法运算溢出检测

$$\text{Overflow} = X_n Y_n \bar{S}_n + \bar{X}_n \bar{Y}_n S_n$$





5.1 定点数加/减运算

5. 补码加减法运算溢出检测

2) 方法2：通过运算中最高位数据的进位与符号位的进位是否同步来检测

假符号位进位为 C_f ，最高数据位进位为 C_n

$C_f=0 \quad C_n=0$

		0	1	0	1	0	1
+		0	0	1	0	0	0
	0	←	0	←			
		0	1	1	1	0	1

$C_f=1 \quad C_n=1$

		1	1	0	1	0	1
+		1	1	1	0	0	0
	1	←	1	←			
	1	1	0	1	1	0	1



5.1 定点数加/减运算

5. 补码加减法运算溢出检测

2) 方法2: 通过运算中最高位数据的进位与符号位的进位是否同步来检测
设符号位进位为 C_f , 最高数据位进位为 C_n

不同步就出现
溢出

$C_f=0 \quad C_n=1$

$C_f=1 \quad C_n=0$

0

1

0

1

0

1

+

0

1

1

0

0

0

0

1

1

0

1

1

0

1

1

0

0

1

0

1

+

1

1

1

0

0

0

1

0

1

0

1

1

1

0

1

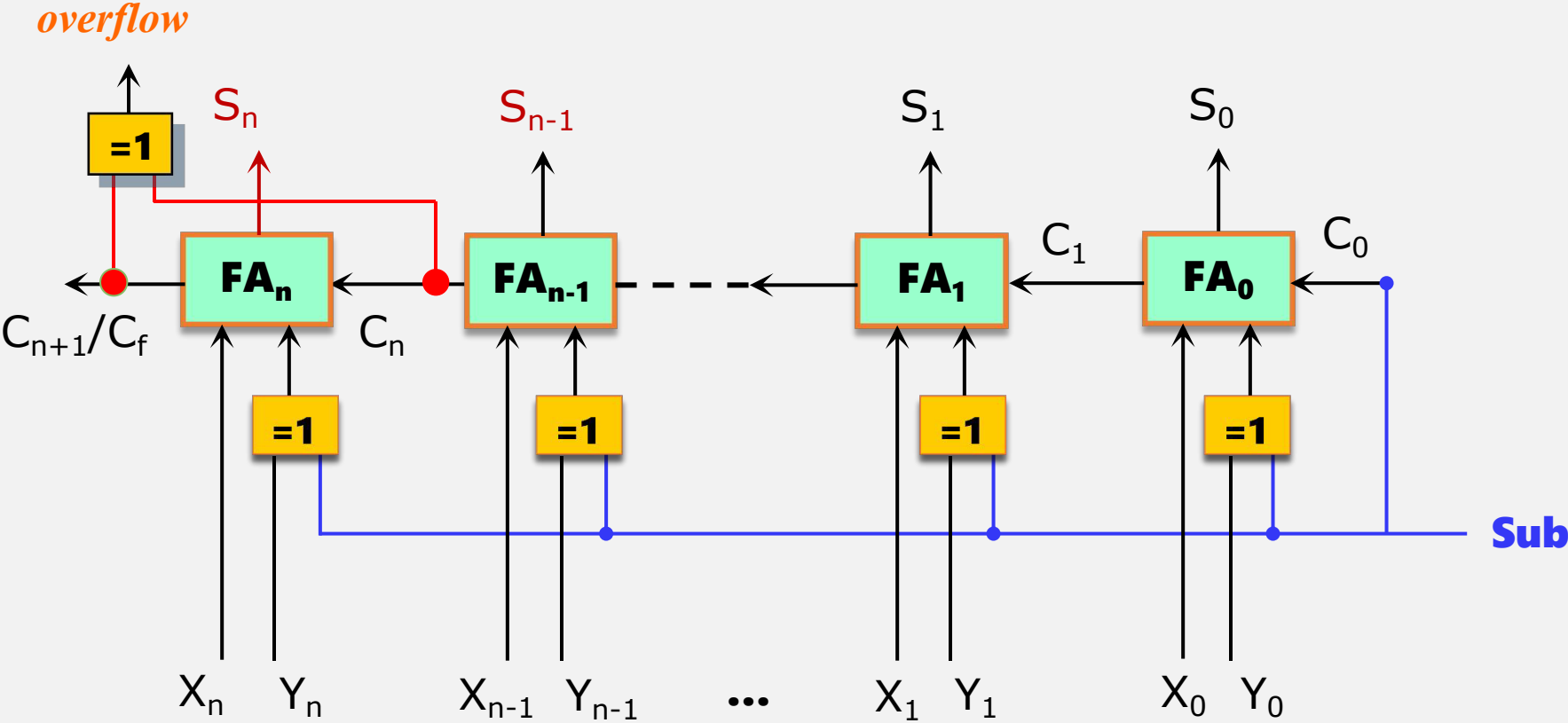
$Overflow = C_f \oplus C_n$



5.1 定点数加/减运算

5. 补码加减法运算溢出检测

方法2: $Overflow = C_f \oplus C_n$





5.1 定点数加/减运算

5. 补码加减法运算溢出检测

3) 方法3: 用变型补码的双符号位, 设双符号位为 $f_1 f_2$, 其中 f_1 为最高符号位

0010101

+

0001000

0011101

$f_1=0 \quad f_2=0$

1110101

+

1111000

11101101

$f_1=1 \quad f_2=1$



5.1 定点数加/减运算

5. 补码加减法运算溢出检测

3) 方法3: 用变型补码的双符号位, 设双符号位为 $f_1 f_2$, 其中 f_1 为最高符号位

双符号位不同就出事

最高符号位为
本数的符号

0 0 1 0 1 0 1

+

0 0 1 1 0 0 0

0 1 0 1 1 0 1

$f_1=0$ $f_2=1$ 正溢出(上溢)

1 1 0 0 1 0 1

+

1 1 1 1 0 0 0

1 1 0 1 1 1 0 1

$f_1=1$ $f_2=0$ 负溢出(下溢)

$Overflow = f_1 \oplus f_2$

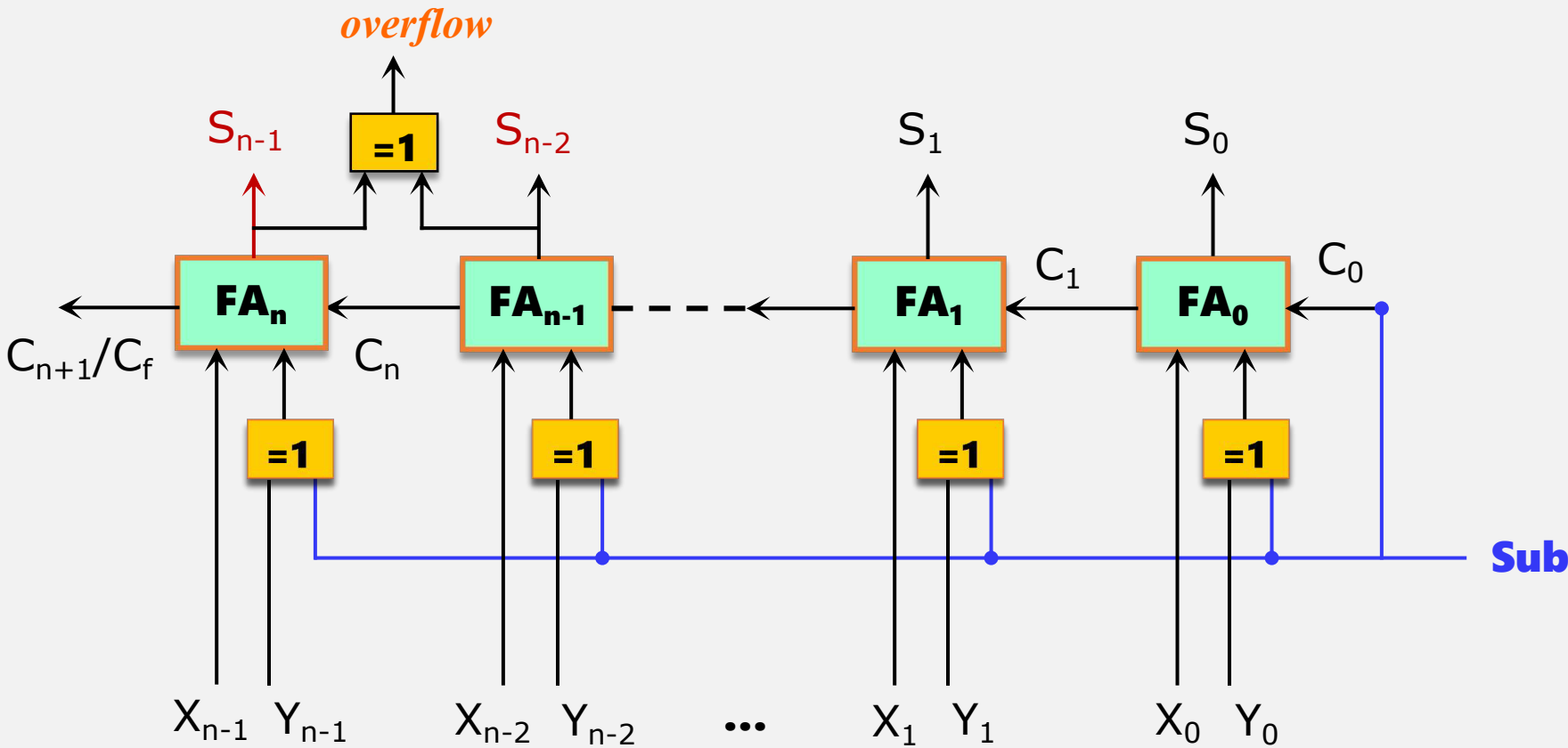
第一符号位是结果的正确符号位



5.1 定点数加/减运算

5. 补码加减法运算溢出检测

方法3 : $Overflow = f_1 \oplus f_2$

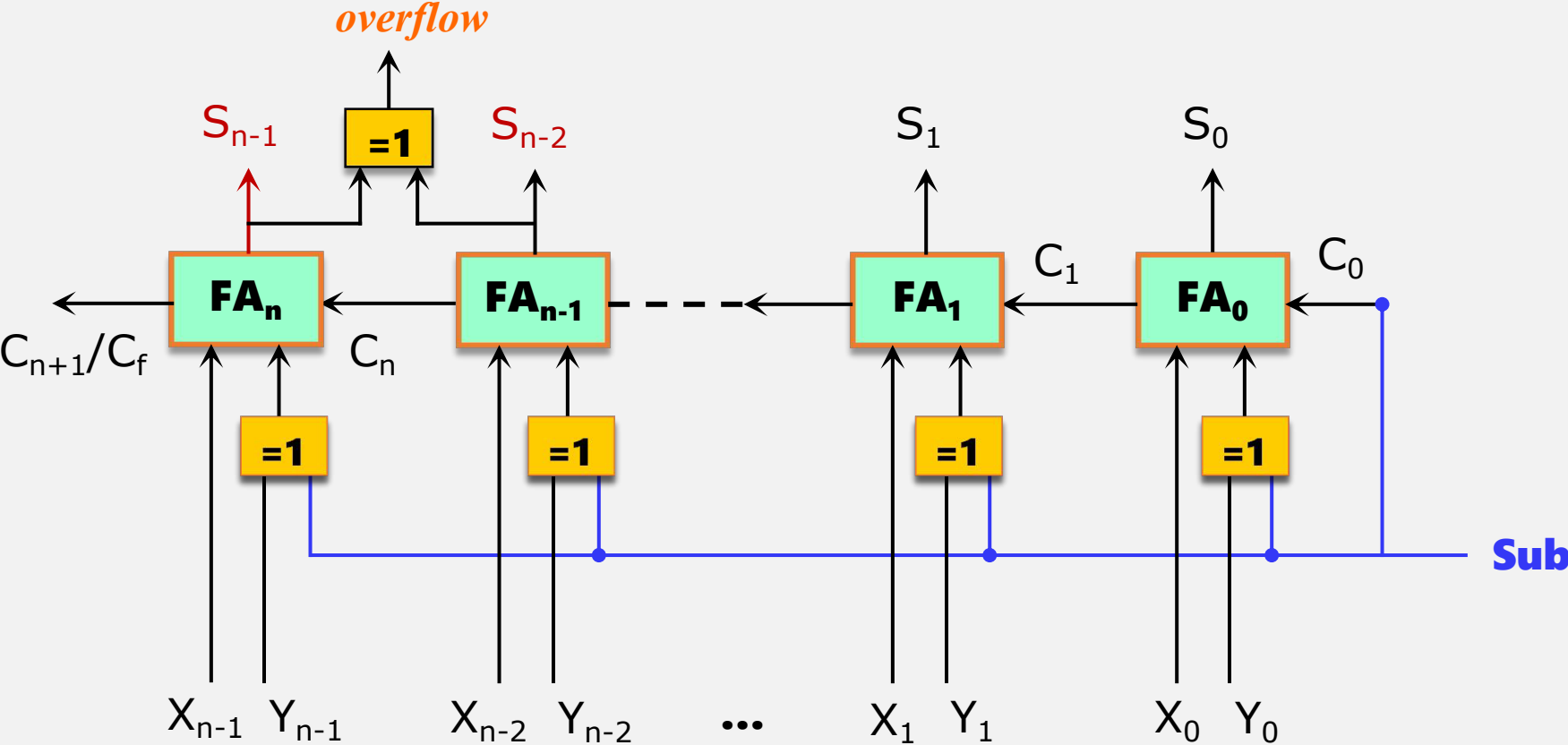




5.1 定点数加/减运算

5. 补码加减法运算溢出检测

方法3 : $Overflow = f_1 \oplus f_2$





5.1 定点数加/减运算

6. 无符号数加减法运算溢出检测

	0	0	0	0	0	1	0	1	3
+	0	0	0	0	1	0	0	0	8
	0	0	0	0	1	1	0	1	11

加法, 未溢出 $C_{n+1}=0$

	1	1	1	1	1	1	1	1	255	
+	1	1	1	1	1	1	1	0	254	
	1	1	1	1	1	1	1	0	1	253

加法 溢出 $C_{n+1}=1$ 且和数小于加数

无符号溢出检测 = $SUB \oplus C_{n+1}$



5.1 定点数加/减运算

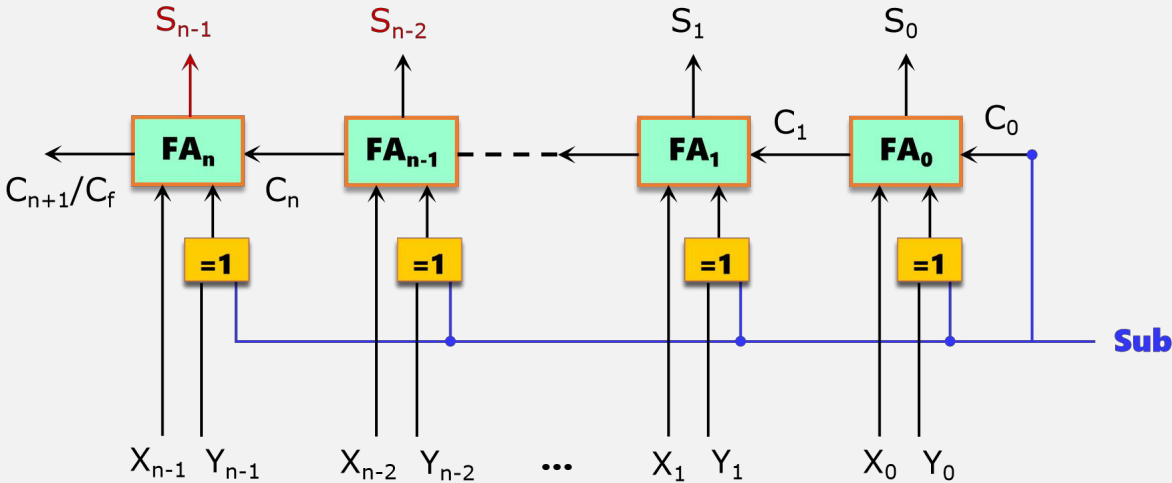
6. 无符号数加减法运算溢出检测

	0	0	0	0	0	1	0	1	5
+	1	1	1	1	0	1	1	0	-10
	0	1	1	1	1	1	0	1	251

减法, 溢出 Cout=0 差大于被减数

	0	0	0	0	1	0	1	0	10
+	1	1	1	1	1	0	1	1	-5
	1	0	0	0	0	0	1	0	5

减法, 未溢出 Cout=1



无符号溢出检测 = $SUB \oplus C_{n+1}$



5.1 定点数加/减运算

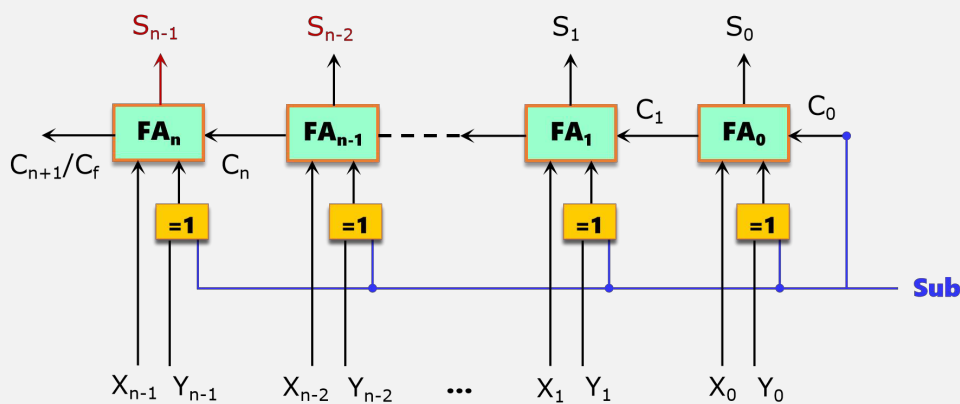
6. 无符号数加减法运算溢出检测

0 1 0 0 0 0 0 1
+ 0 1 0 0 0 0 1 0

0 1 0 0 0 0 0 1 1

- ◆ 无符号数加，65+ 66 = 131，正常
- ◆ 有符号数加，65+ 66= -125，溢出
- ◆ 无符号数减，65- 190 = 131，溢出
- ◆ 有符号数减，65-(-66) = -125, 溢出

无符号溢出检测 = $SUB \oplus C_{n+1}$





5.1 定点数加/减运算

6. 无符号数加减法运算溢出检测

ISA	无符号运算	有符号运算	浮点运算
X86	ADD/SUB 加减		FADD/FSUB 加减
	MUL/DIV 乘除	IMUL/IDIV 乘除	FMUL/FDIV 乘除
MIPS32	ADDU/SUBU 加减	ADD/SUB 加减	ADD.S ADD.D / SUB.S SUB.D 加减
	MULTU/DIVU 乘除	MULT/DIV 乘除	MUL.S MUL.D / DIV.S DIV.D 乘除
RISC-V32	ADD/SUB 加减		FADD.S FADD.D / FSUB.S FSUB.D 加减
	MULHU/DIVU 乘除	MULH/DIV 乘除	FMUL.S FMUL.D / FDIV.S FDIV.D 乘除

7. 补码加减法运算溢出检测



■ ■ ■ ■ ■

ADD AX, BX

JO LOOP

■ ■ ■ ■ ■ ■

LOOP:

■ ■ ■ ■ ■ ■ ■ ■ ■ ■

7. 补码加减法运算溢出检测

问题 1 在你过去的C语言程序设计中考虑过溢出检测吗？

问题 2 若CPU中未设计上述溢出检测电路及标志寄存器，如何检测溢出？

```
int tadd_ok(int x,int y)
{
    int sum= x+y;
    int neg_over= x<0&& y<0&&sum >=0;
    int pos_over= x>=0&& y>=0&&sum<0;
    return !neg_over&&!pos_over; }
```

- ◆ 体会软硬溢出检测的逻辑等效性
- ◆ 体会软硬溢出检测的差异性
- ◆ 对你未来的程序设计有何启示？
- ◆ 这就是软硬协同的系统观



5.1 定点数加/减运算

7. 补码加减法运算溢出检测

问题：高级语言中如何利用CPU设计的溢出检测电路和标志寄存器检测溢出？

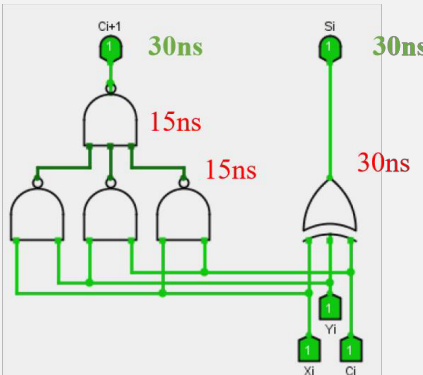
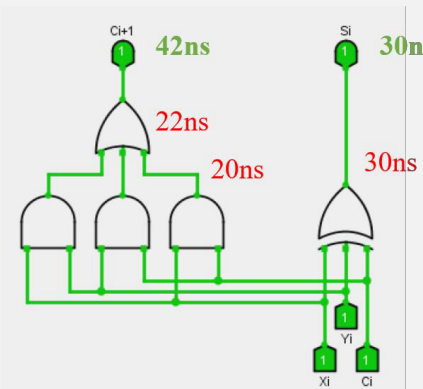
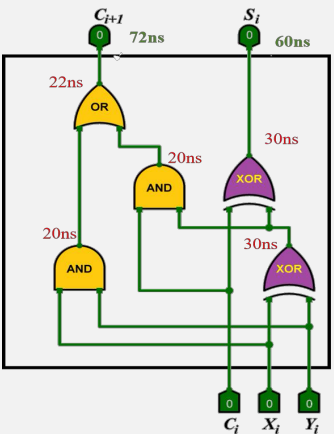
https://blog.csdn.net/lwx62/article/details/82796364?spm=1001.2101.3001.6650.1&utm_medium=distribute.pc_relevant.none-task-blog-2%7Edefault%7ECTRLIST%7ERate-1-82796364-blog-117238051.235%5Ev27%5Epc_relevant_3mothn_strategy_recovery&depth_1-utm_source=distribute.pc_relevant.none-task-blog-2%7Edefault%7ECTRLIST%7ERate-1-82796364-blog-117238051.235%5Ev27%5Epc_relevant_3mothn_strategy_recovery&utm_relevant_index=2

GCC内联汇编教程

8. 快速加法器设计

$$S_i = X_i \oplus Y_i \oplus C_i$$

$$C_{i+1} = X_i Y_i + (X_i \oplus Y_i) C_i$$



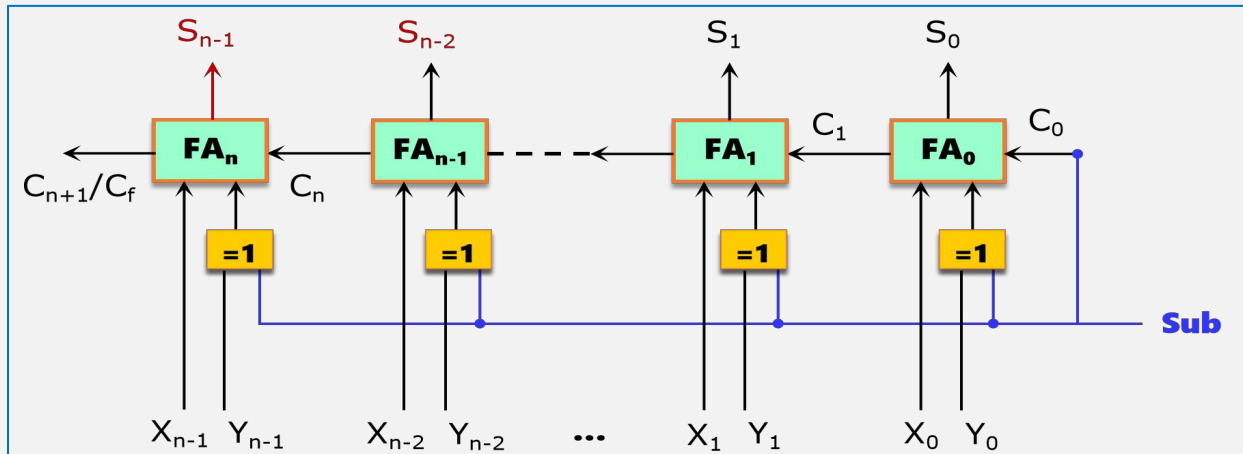
$$G_i = X_i Y_i$$

进位生成函数 **Generate**

$$P_i = X_i \oplus Y_i$$

进位传递函数 **Propagate**

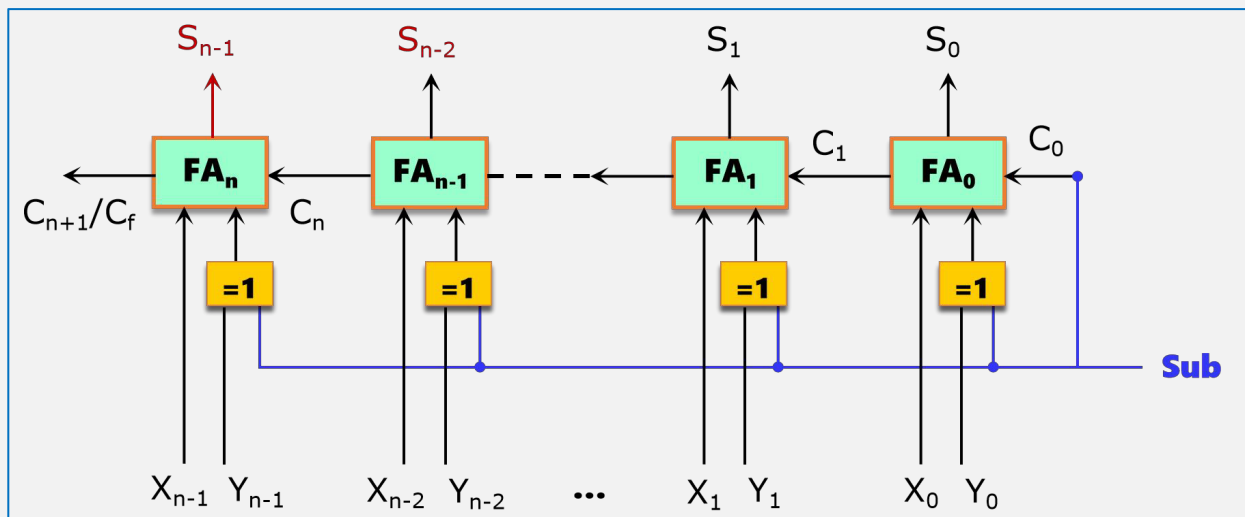
$$C_{i+1} = G_i + P_i C_i$$



C_{n+1} 时延: $(n-1)$ 单FA进位时延 t
问题: 时延很大

5.1 定点数加/减运算

8. 快速加法器设计



$$G_i = X_i Y_i$$

$$P_i = X_i \oplus Y_i$$

$$C_{i+1} = G_i + P_i C_i$$

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 C_1$$

$$C_3 = G_2 + P_2 C_2$$

.....

$$C_n = G_{n-1} + P_{n-1} C_{n-1}$$

$$C_{n+1} = G_n + P_n C_n$$

高位依赖于低位进位,形成了串行进位链

8. 快速加法器设计

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 C_1$$

$$= G_1 + P_1(G_0 + P_0 C_0) = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 C_2$$

$$= G_2 + P_2(G_1 + P_1 G_0 + P_1 P_0 C_0)$$

$$= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

$$G_i = X_i Y_i$$

$$P_i = X_i \oplus Y_i$$

$$C_{i+1} = G_i + P_i C_i$$

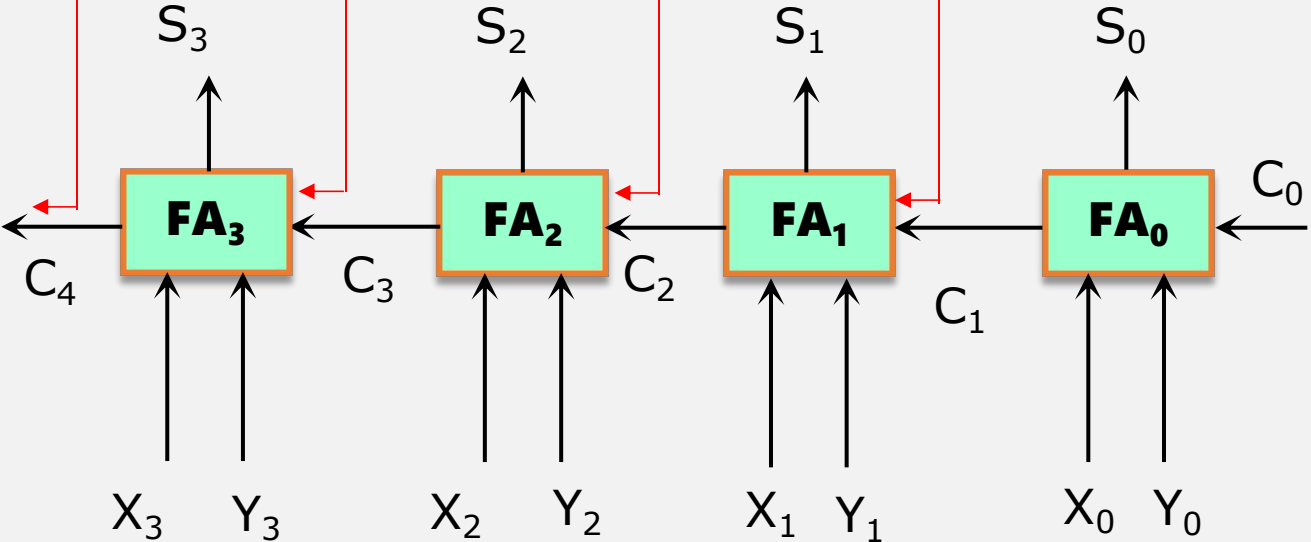
8. 快速加法器设计

$$C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$$

$$C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$$

$$C_2 = G_1 + P_1(G_0 + P_0C_0) = G_1 + P_1G_0 + P_1P_0C_0$$

$$C_1 = G_0 + P_0C_0$$



5.1 定点数加/减运算

8. 快速加法器设计

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

$$C_{16} = G_{15} + P_{15} G_{14} + \cdots + P_{15} P_{14} \cdots P_1 P_0 C_0$$

17个输入端的与门



扇入系数：逻辑门允许的输入端数量
(1~8)

扇出系数：逻辑门输出端所能驱动的同类逻辑门的个数，也称负载能力

TTL(Transistor-Transistor Logic)电路的扇出系数 N_c 为8 ~ 10。

CMOS(Complementary Metal-Oxide-Semiconductor)电路的扇出系数 N_c 可达20 ~ 25

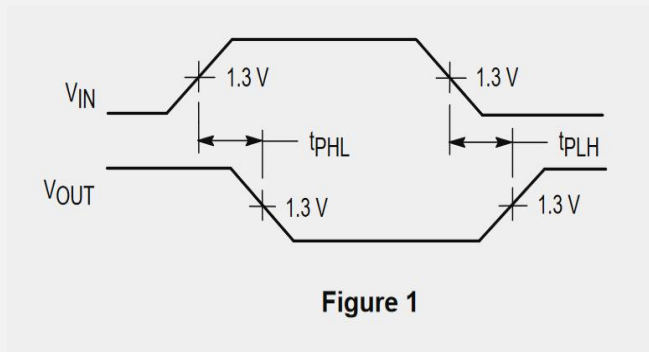
每一个小电流不能汇集
为一大电流

8. 快速加法器设计

! 门电路输入端的增加，会使串联的MOS管的总的等效电阻增加，会使输出电压偏离电源电压或地电平。

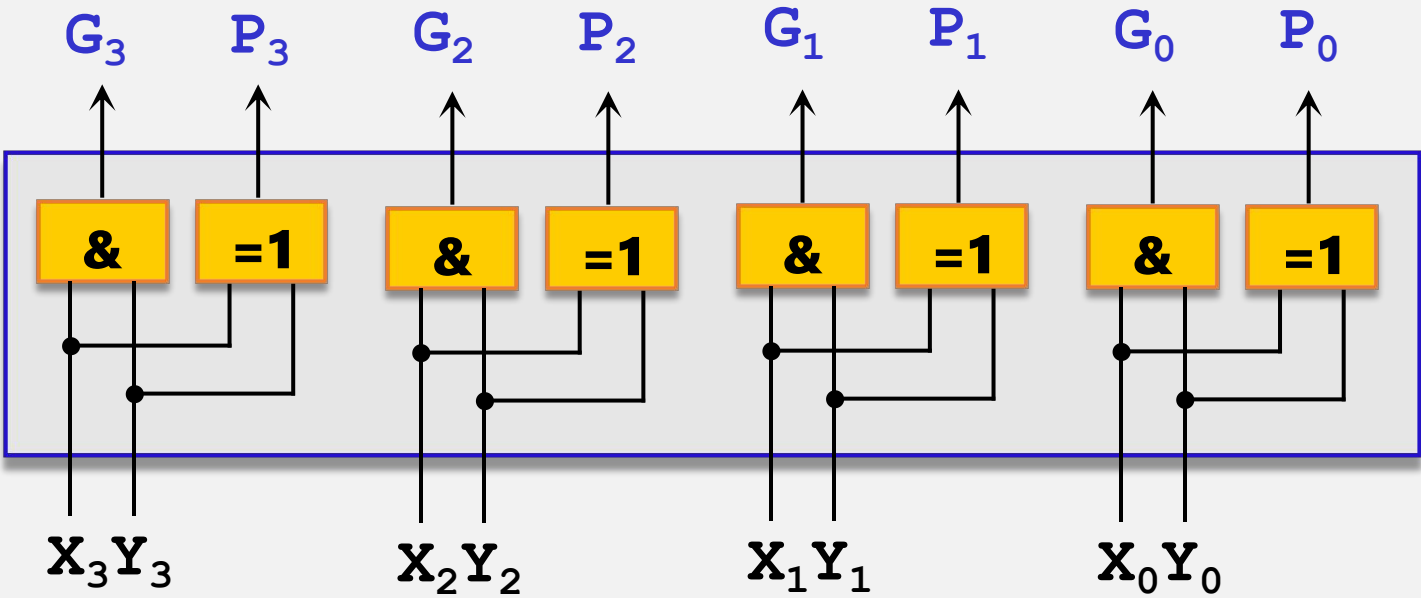
! 多扇出问题，会严重影响FPGA布线的稳定性，不利于时序收敛
当输出负载大于它的扇出能力的时候，就会有如下的效应：

- 1) 输出低态时，输出电压 V_{OL} 可能高于 V_{OLmax} 。
- 2) 输出高态时，输出电压 V_{OH} 可能高于 V_{OLmin} 。
- 3) 输出的传播延迟可能大于规格说明的延迟值。
- 4) 输出的上升和下降时间可能大于规格说明的延迟值。
- 5) 器件工作温度可能升高，从而降低其可靠性，最终引起器件失效。



8. 快速加法器设计

先做4位

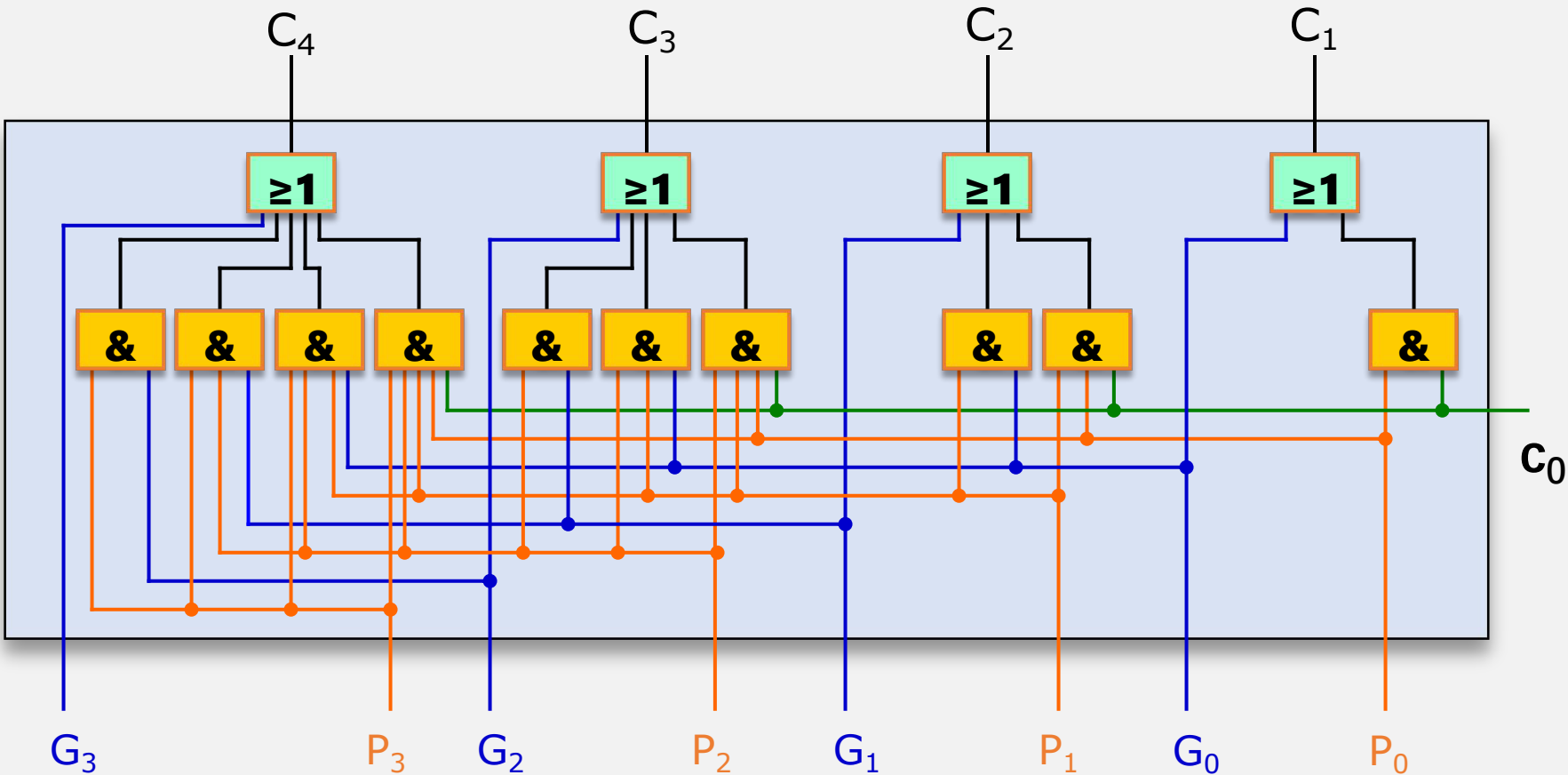


$$G_i = X_i Y_i$$

$$P_i = X_i \oplus Y_i$$

8. 快速加法器设计

$$C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$$



8. 快速加法器设计

$$G_i = X_i Y_i$$

$$P_i = X_i \oplus Y_i$$

$$S_3 = X_3 \oplus Y_3 \oplus C_3 = P_3 \oplus C_3$$

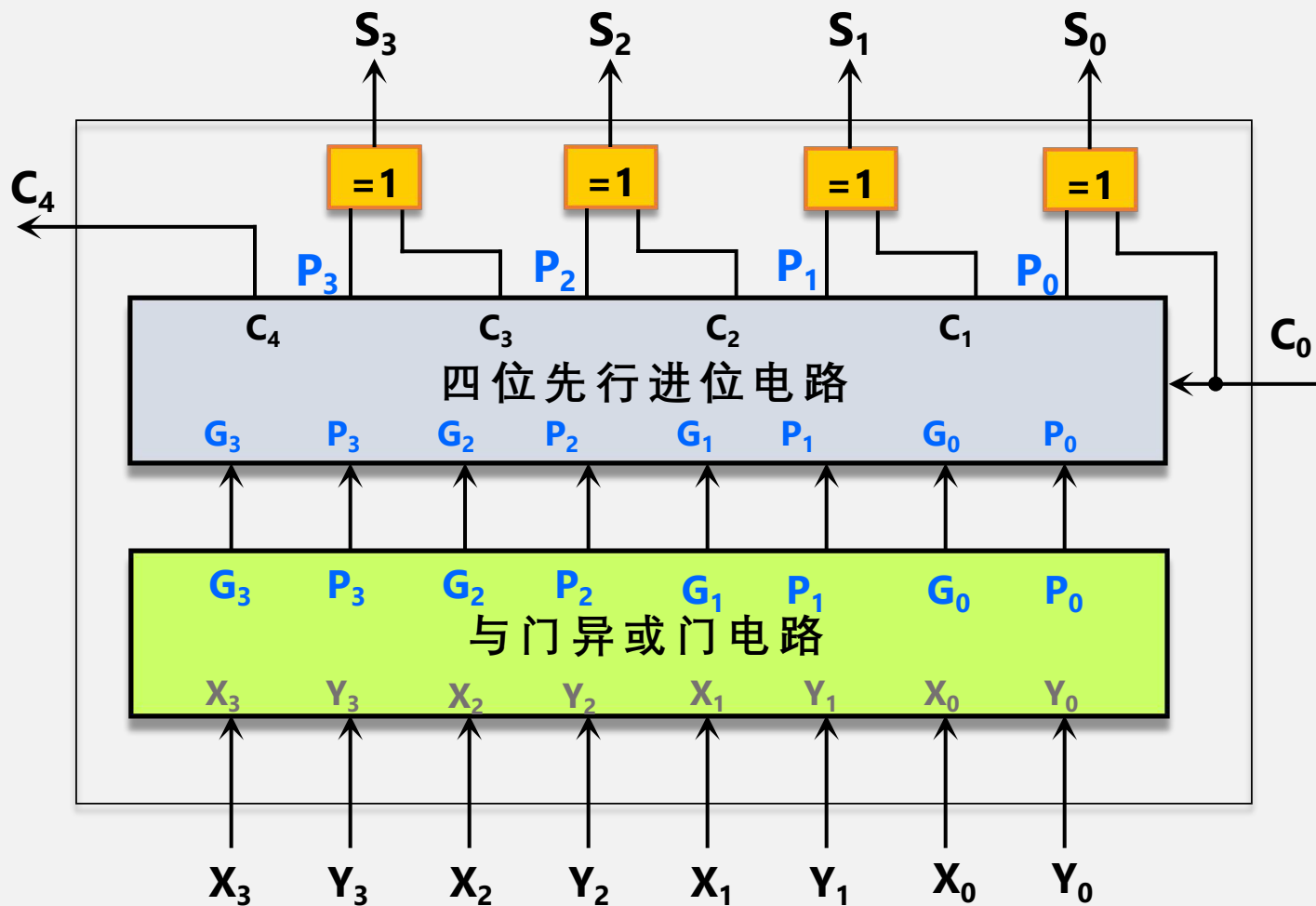
$$S_2 = X_2 \oplus Y_2 \oplus C_2 = P_2 \oplus C_2$$

$$S_1 = X_1 \oplus Y_1 \oplus C_1 = P_1 \oplus C_1$$

$$S_0 = X_0 \oplus Y_0 \oplus C_0 = P_0 \oplus C_0$$

5.1 定点数加/减运算

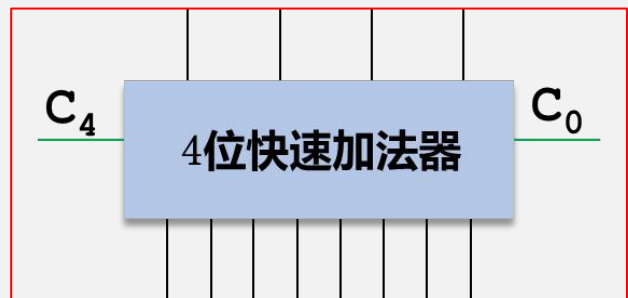
8. 快速加法器设计



$$S_3 = X_3 \oplus Y_3 \oplus C_3 = P_3 \oplus C_3$$

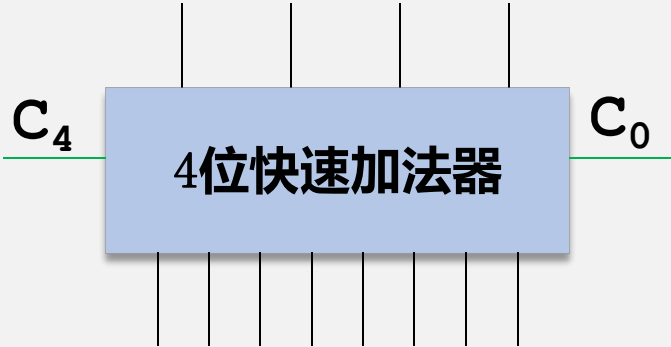
$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$P_i = X_i \oplus Y_i \quad G_i = X_i Y_i$$

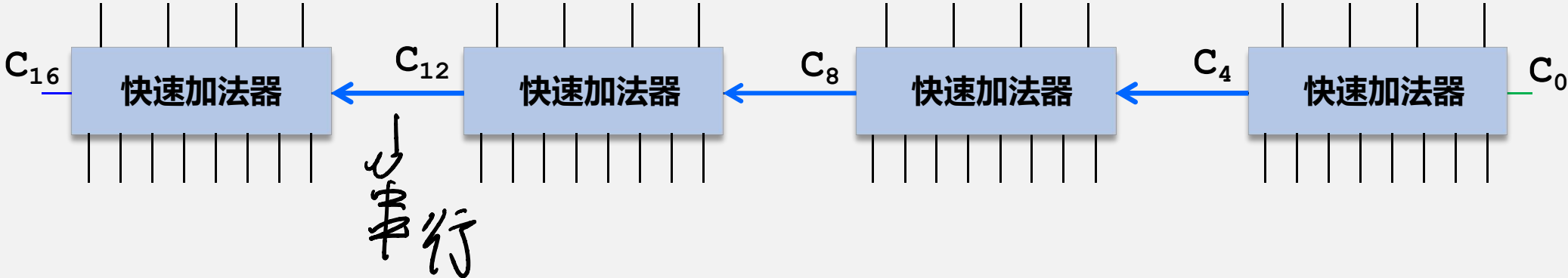


5.1 定点数加/减运算

8. 快速加法器设计



- ◆ 组内先行进位
- ◆ 组间 ? 串行进位
- ◆ 可否组间并行?



5.1 定点数加/减运算

8. 快速加法器设计

$$C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$$

也是 C_8 , 只是 C_0 不同

$$G^* = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0$$

成组进位生成函数

$$P^* = P_3P_2P_1P_0$$

成组进位传递函数

$$C_4 = G^* + P^*C_0$$

$$C_1 = G_0 + P_0C_0$$

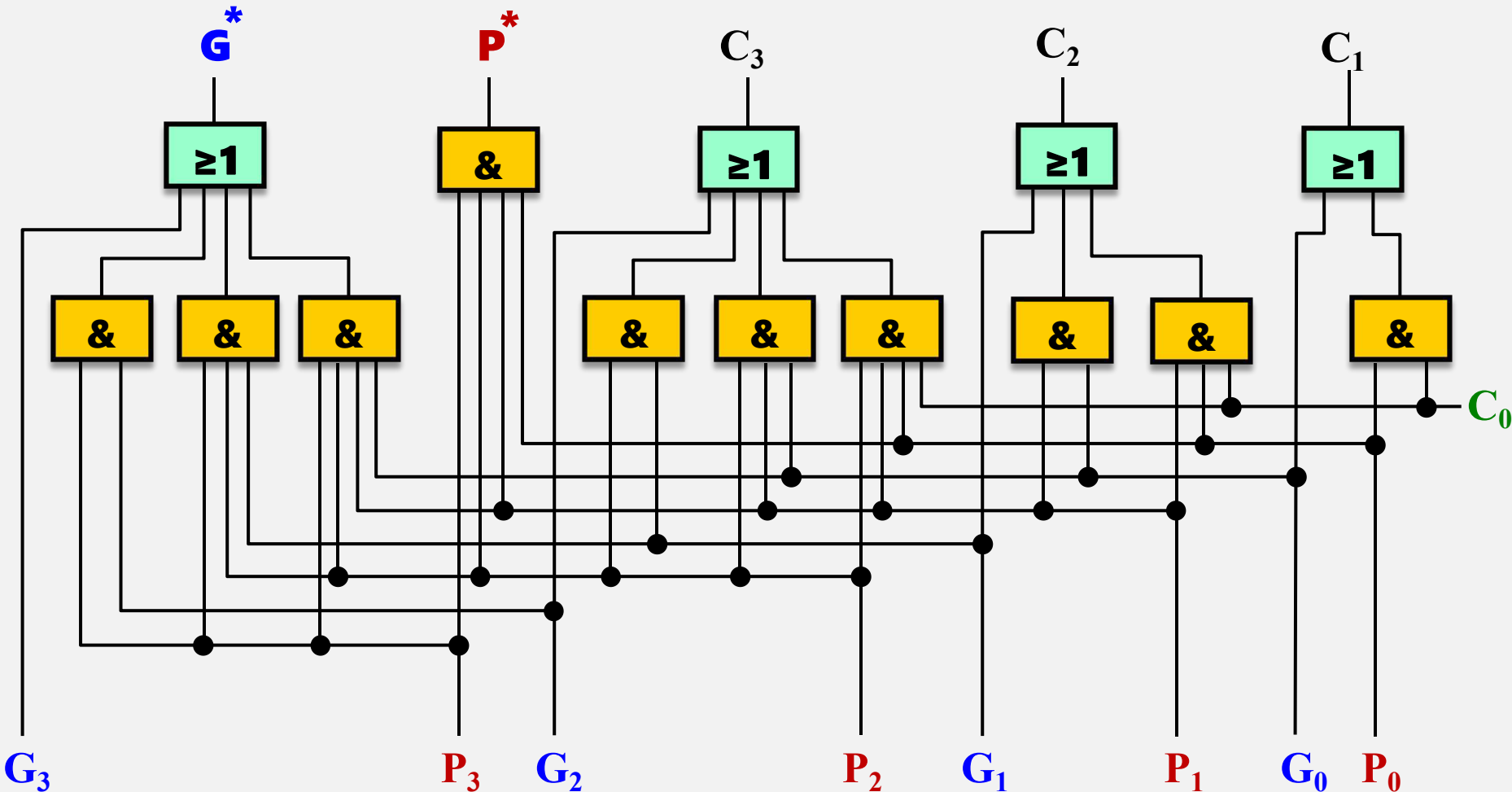
两式逻辑表达式完全一样!

? 也应该完全一样!

同理 可得: C_8 VS C_2 C_{12} VS C_3 , C_{16} VS C_4 的表达式也一样

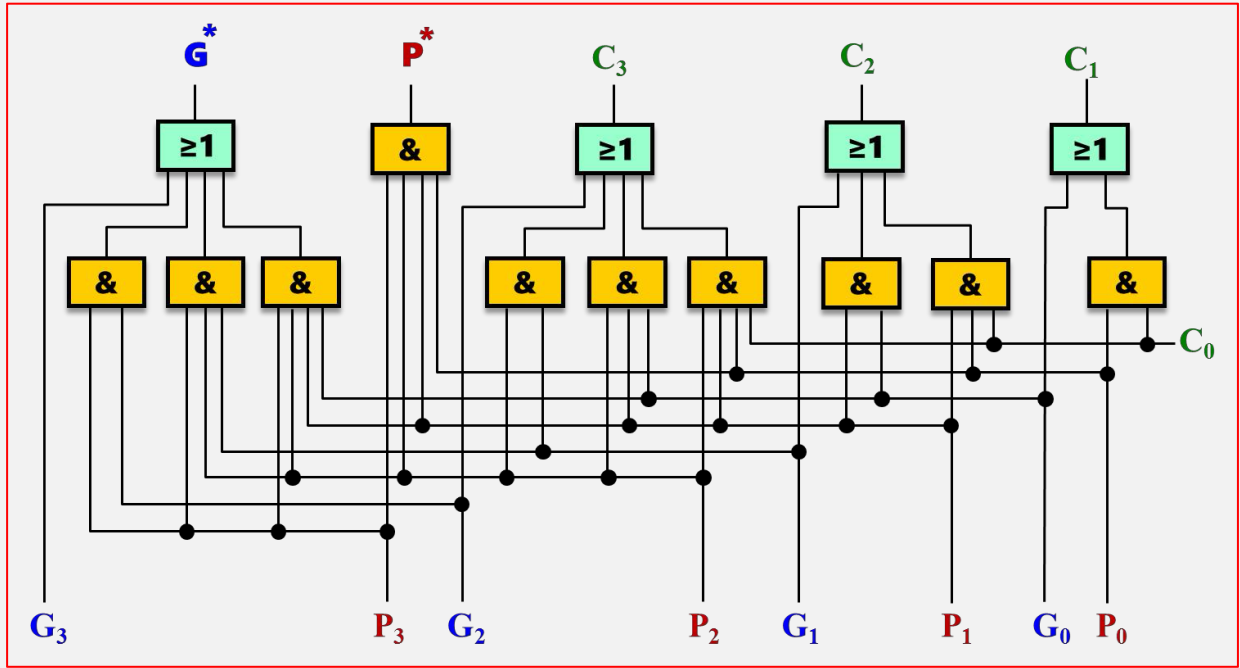
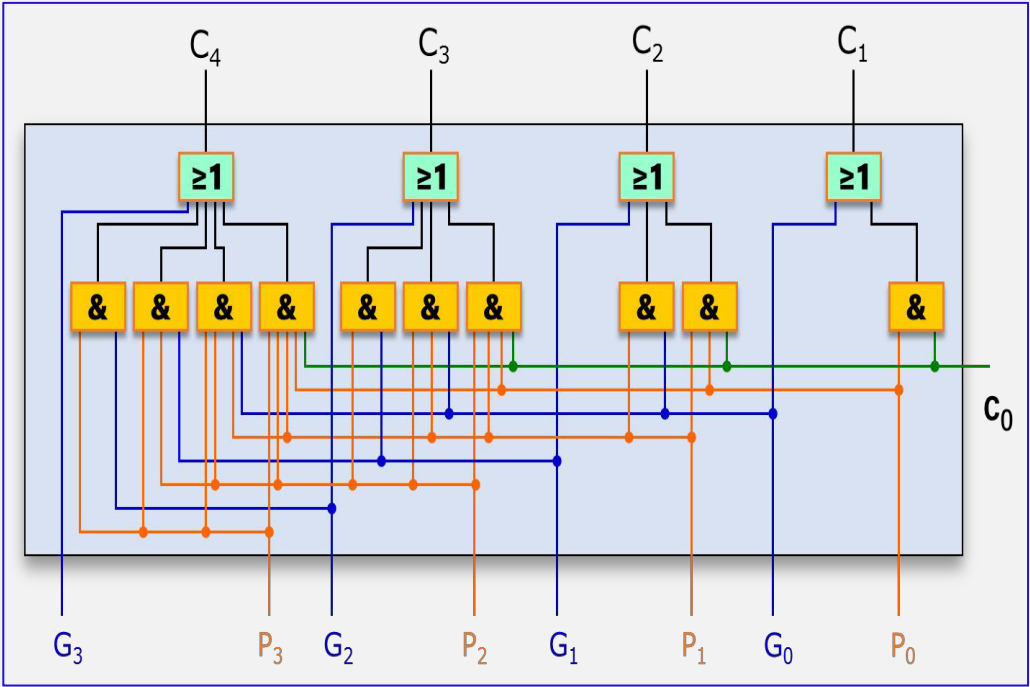
8. 快速加法器设计

两级先行进位电路



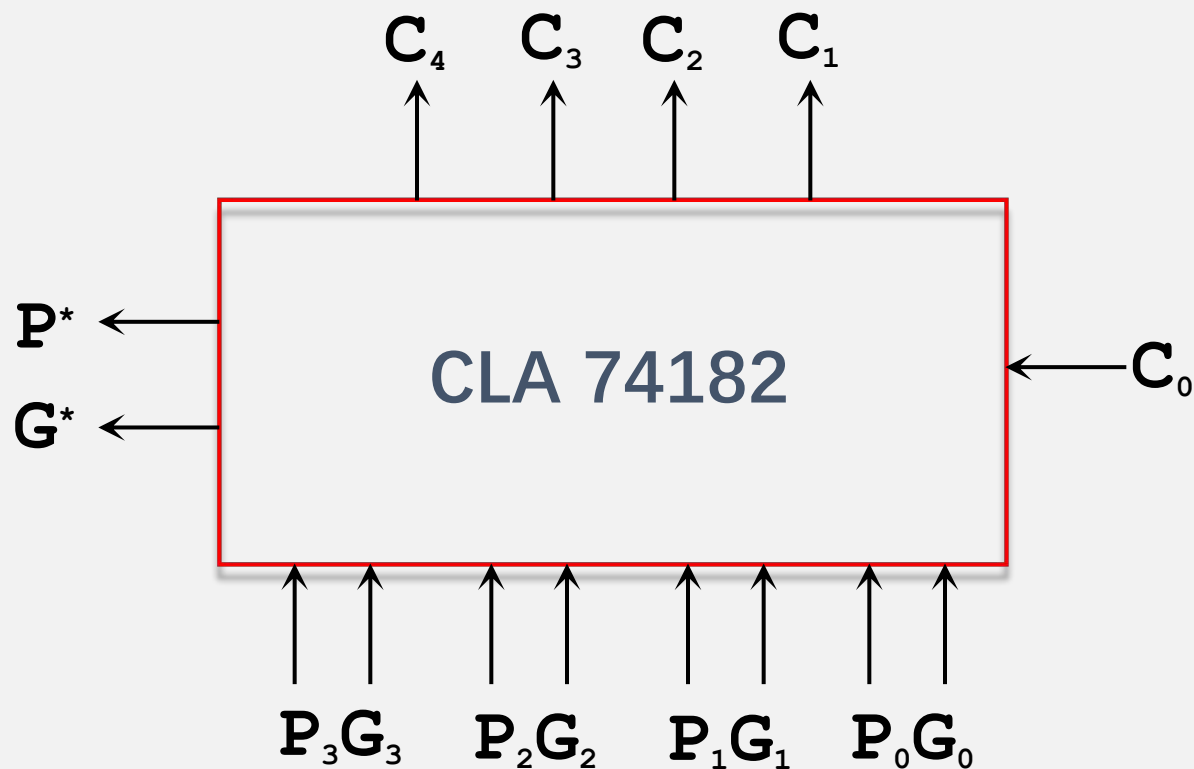
5.1 定点数加/减运算

8. 快速加法器设计



5.1 定点数加/减运算

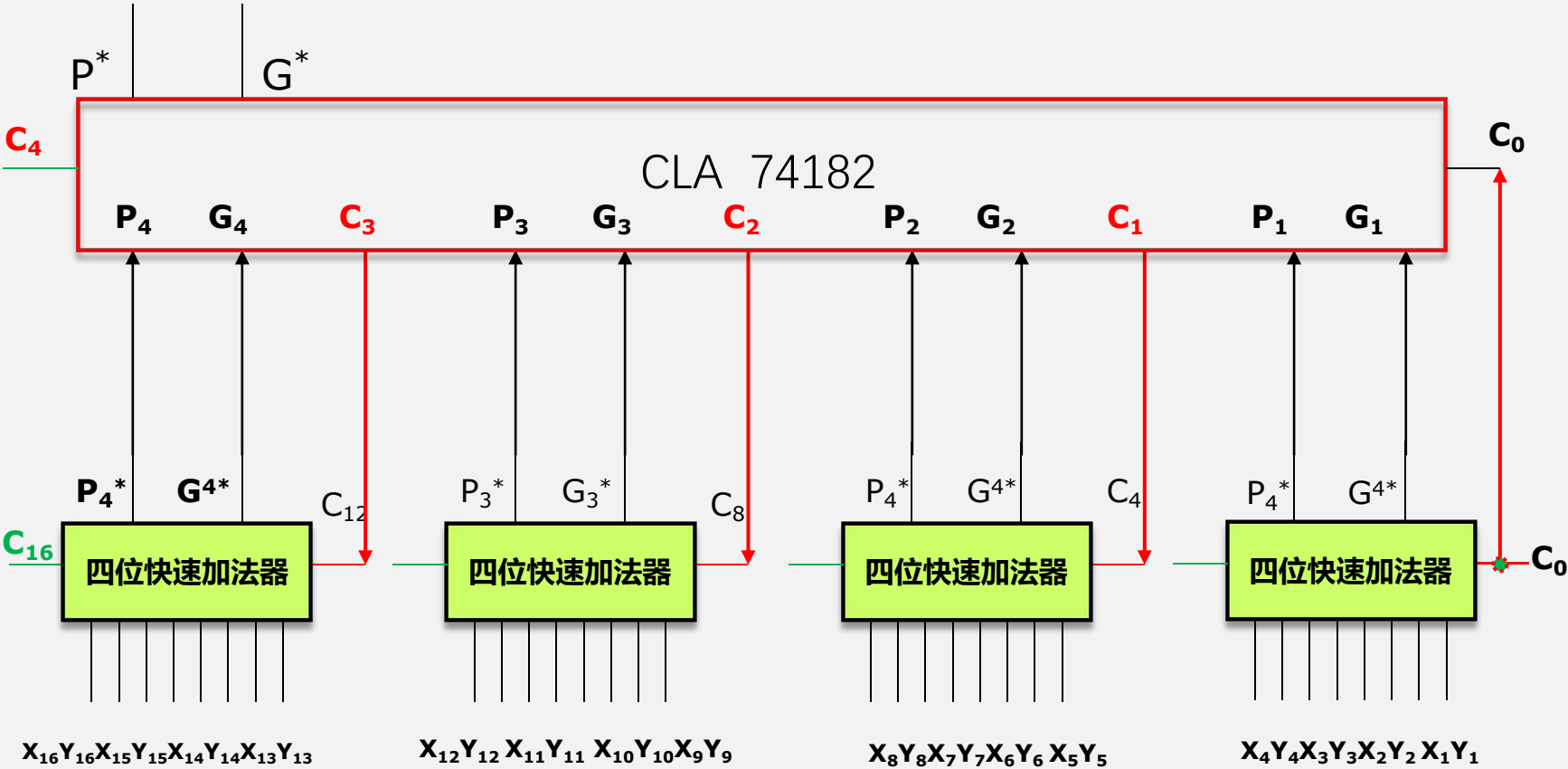
8. 快速加法器设计





5.1 定点数加/减运算

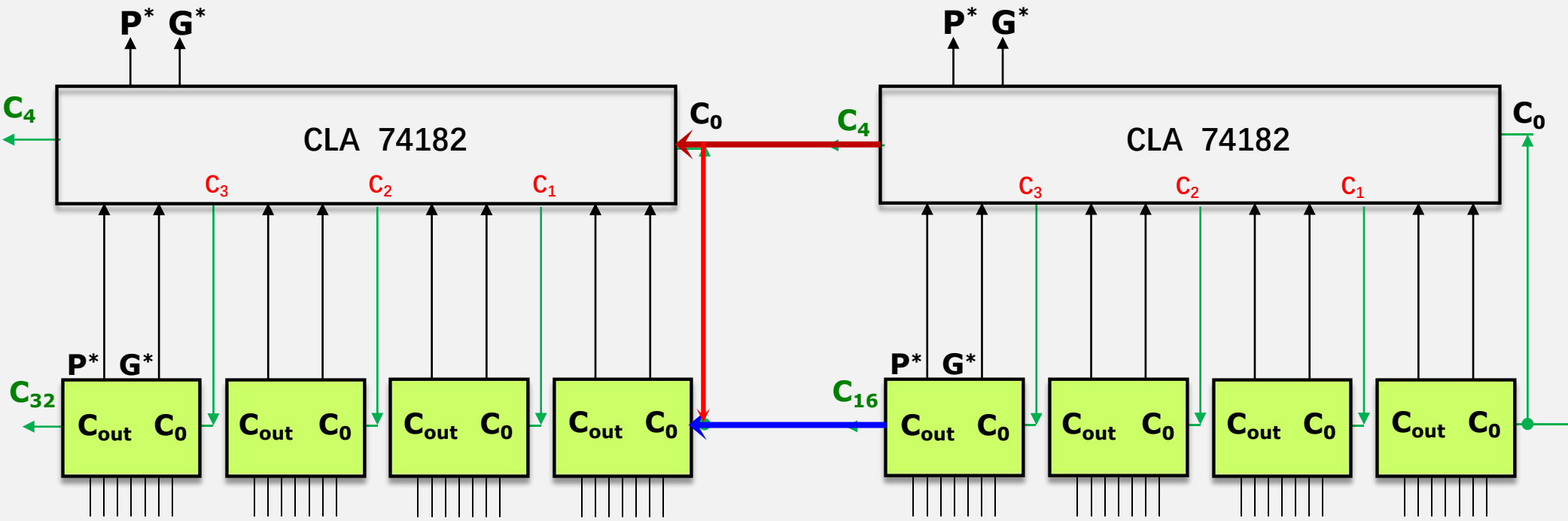
8. 快速加法器设计





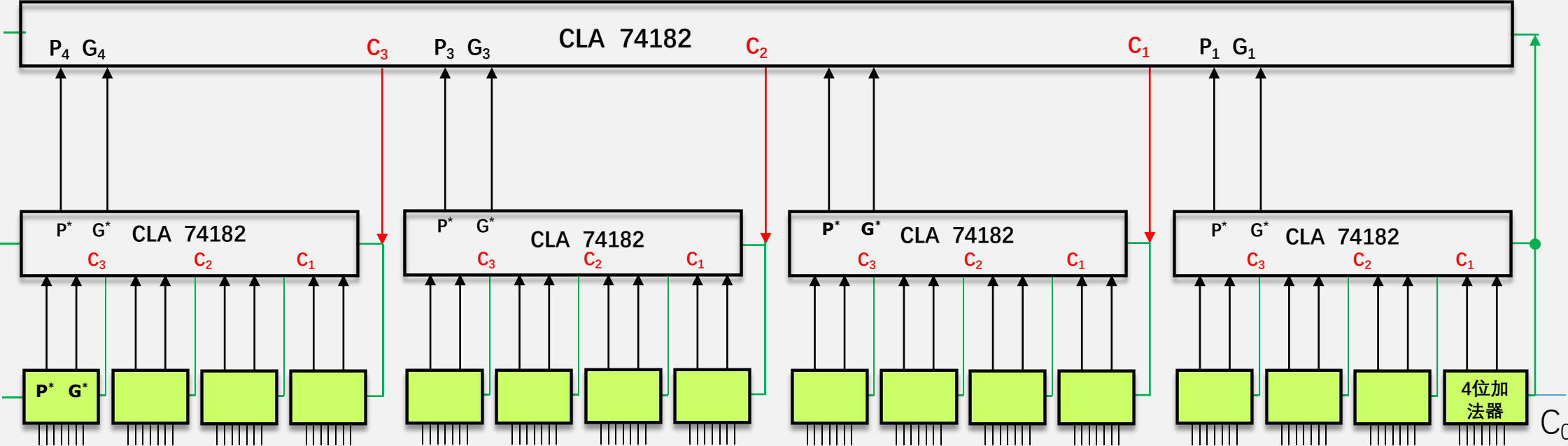
5.1 定点数加/减运算

8. 快速加法器设计



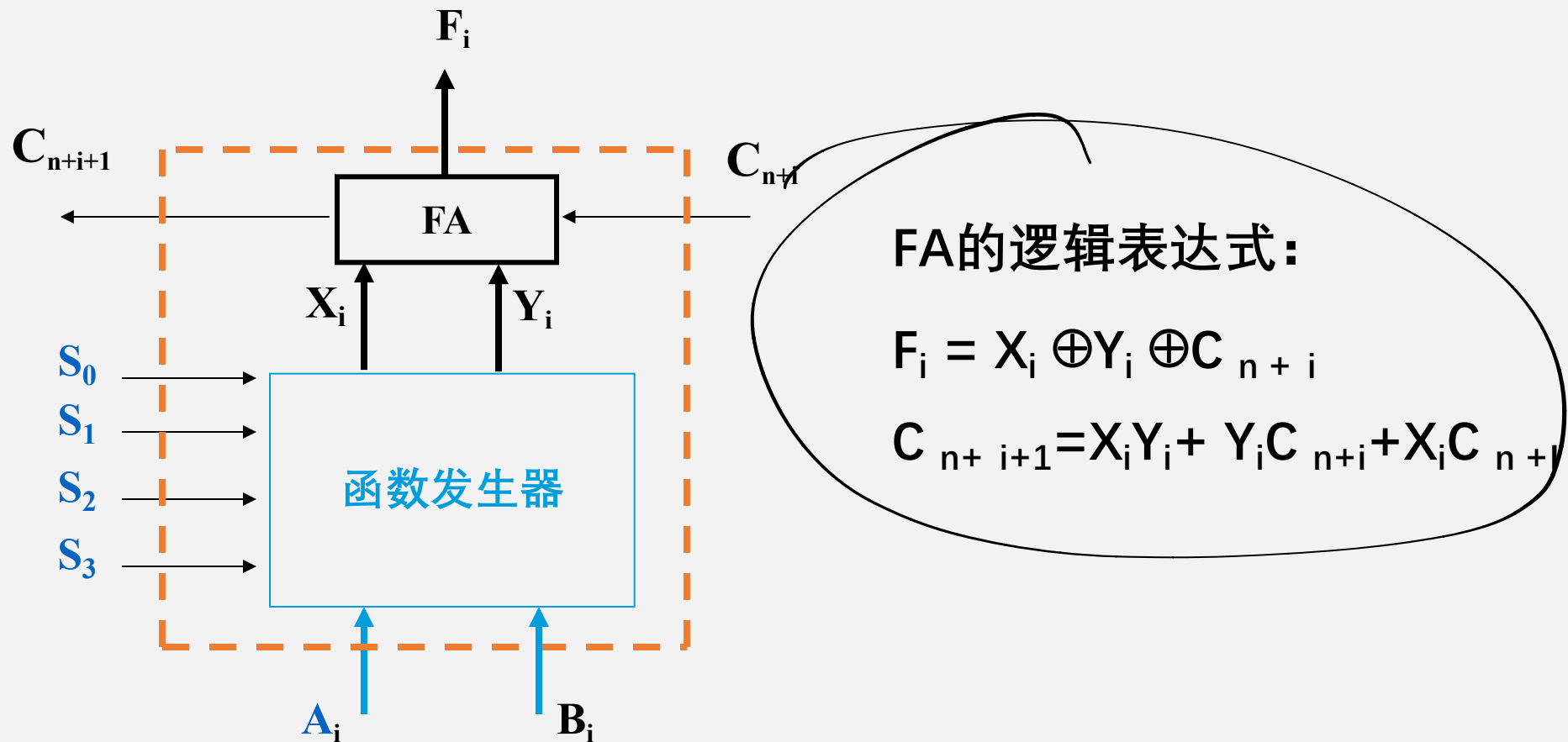


8. 快速加法器设计



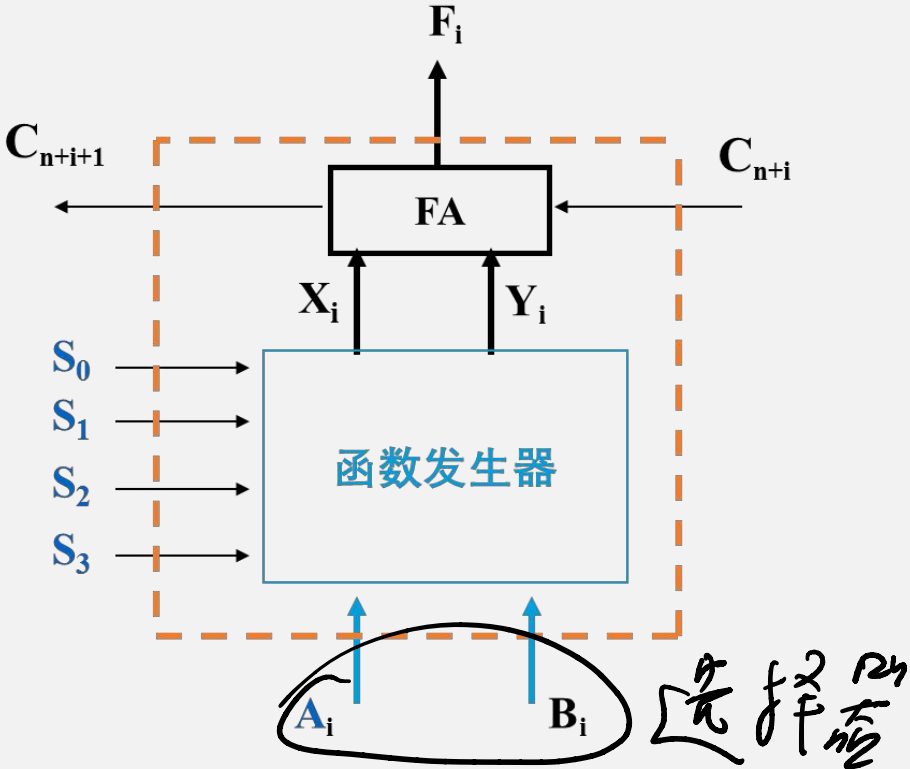
5.1 定点数加/减运算

9. 四位运算器芯片 SN74181



9. 四位运算器芯片 SN74181

S0	S1	Y _i	S2	S3	X _i
0	0	$\overline{A_i}$	0	0	1
0	1	$\overline{A_i}B_i$	0	1	$\overline{A_i}+\overline{B_i}$
1	0	$\overline{A_i} \overline{B_i}$	1	0	$\overline{A_i}+B_i$
1	1	0	1	1	$\overline{A_i}$



$$X_i = \overline{S_2} \overline{S_3} + \overline{S_2} S_3 (\overline{A_i} + \overline{B_i}) + S_2 \overline{S_3} (\overline{A_i} + B_i) + S_2 S_3 \overline{A_i}$$

$$Y_i = \overline{S_0} \overline{S_1} \overline{A_i} + \overline{S_0} S_1 \overline{A_i} B_i + S_0 \overline{S_1} \overline{A_i} \overline{B_i}$$

$$X_i = \overline{S_3 A_i B_i + S_2 A_i \overline{B_i}}$$

$$Y_i = \overline{A_i + S_0 B_i + S_1 \overline{B_i}}$$

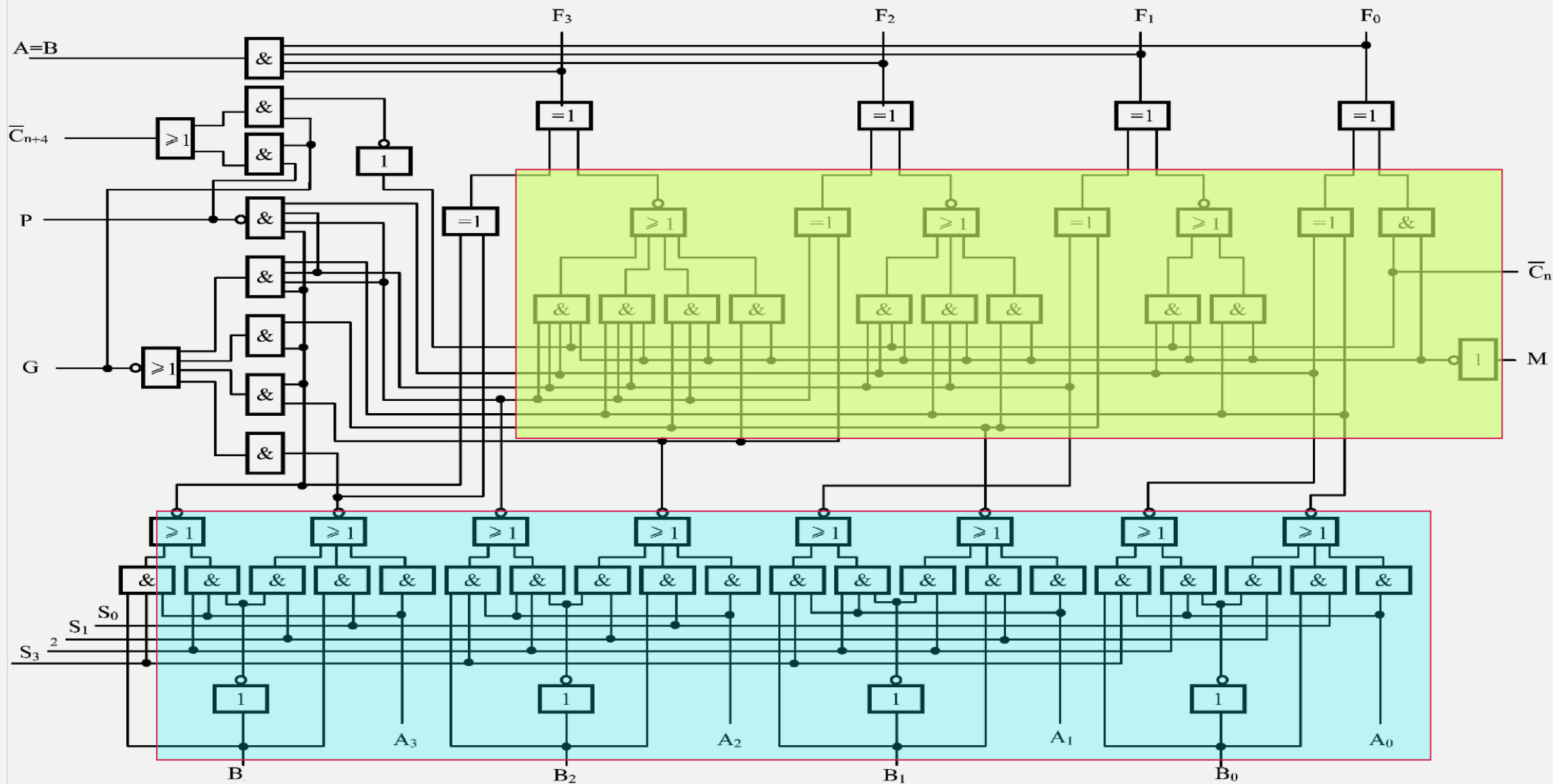
FA的逻辑表达式：

$$F_i = X_i \oplus Y_i \oplus C_{n+i}$$

$$C_{n+i+1} = X_i Y_i + Y_i C_{n+i} + X_i C_{n+i}$$

5.1 定点数加/减运算

9. 四位运算器芯片 SN74181





5.1 定点数加/减运算

9. 四位运算器芯片 SN74181

?

什么意思?

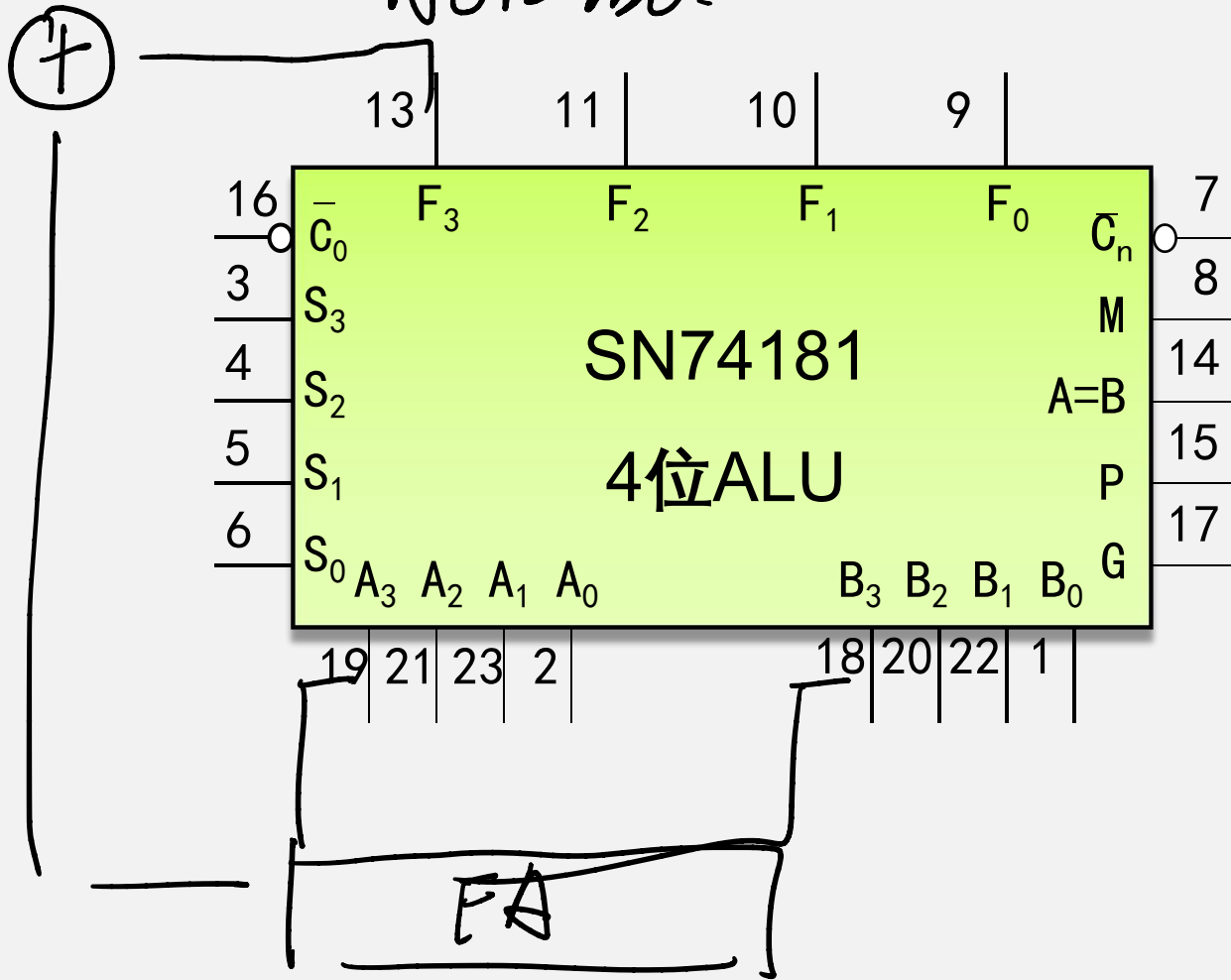
- ◆ 加、减: 算术运算, + : 逻辑运算;
- ◆ M=H时逻辑运算, M=L 时算术运算;
- ◆ 算术运算结果和过程均用补码表示;
- ◆ 当 $S_3S_2S_1S_0=LHHL$, $M=L$, $C=0$ 时, $F = ?$

$S_3S_2S_1S_0$	逻辑运算 $M=H$	算术运算 $M=L$ $C_n=H$
LLLL	$F = \overline{A}$	$F = A$
LLLH	$F = \overline{A + B}$	$F = A + B$
LLHL	$F = \overline{A} B$	$F = A + \overline{B}$
LLHH	$F = \text{逻辑 } 0$	$F = -1$
LHLL	$F = \overline{A} \overline{B}$	$F = A \text{ 加 } A \overline{B}$
LHLH	$F = B$	$F = (A + B) \text{ 加 } A \overline{B}$
LHHL	$F = A \oplus B$	$F = A \text{ 减 } B \text{ 减 } 1$
LHHH	$F = A B$	$F = A B \text{ 减 } 1$
HLLL	$F = \overline{A} + B$	$F = A \text{ 加 } A B$
HLLH	$F = \overline{A} \oplus B$	$F = A \text{ 加 } B$
HLHL	$F = B$	$F = (A + \overline{B}) \text{ 加 } A B$
HLHH	$F = A B$	$F = A B \text{ 减 } 1$
HHLL	$F = 1$	$F = A \text{ 加 } A$
HHLH	$F = A + \overline{B}$	$F = (A + B) \text{ 加 } A$
HHHL	$F = A + B$	$F = (A + \overline{B}) \text{ 加 } A$
HHHH	$F = A$	$F = A \text{ 减 } 1$

5.1 定点数加/减运算

9. 四位运算器芯片 SN74181

简图给出



前面讲过的三种溢出检测方法可直接用于74181吗?

$$\text{Overflow}_1 = X_n Y_n \overline{S_n} + \overline{X_n} \overline{Y_n} S_n$$

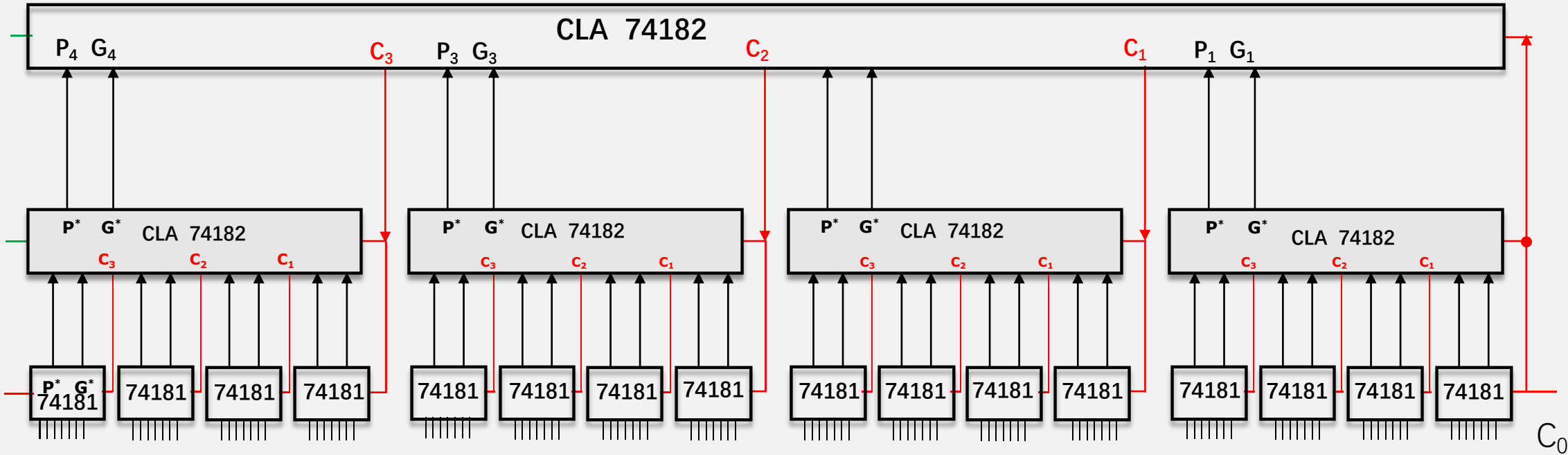
$$\text{Overflow}_2 = C_f \oplus C_n$$

$$\text{Overflow}_3 = f_1 \oplus f_2$$



5.1 定点数加/减运算

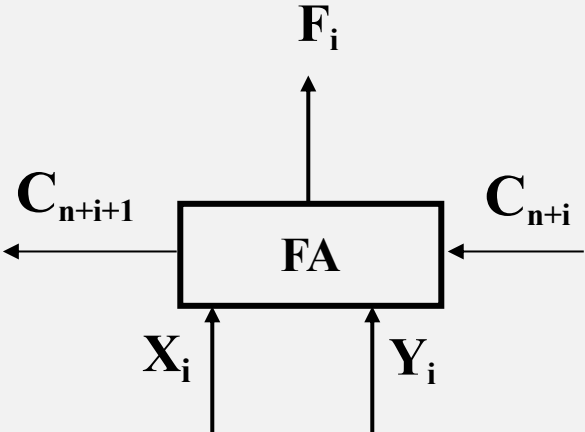
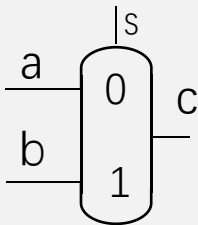
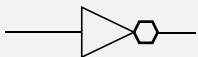
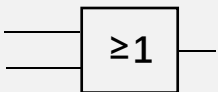
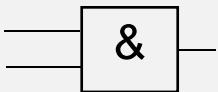
9. 四位运算器芯片 SN74181





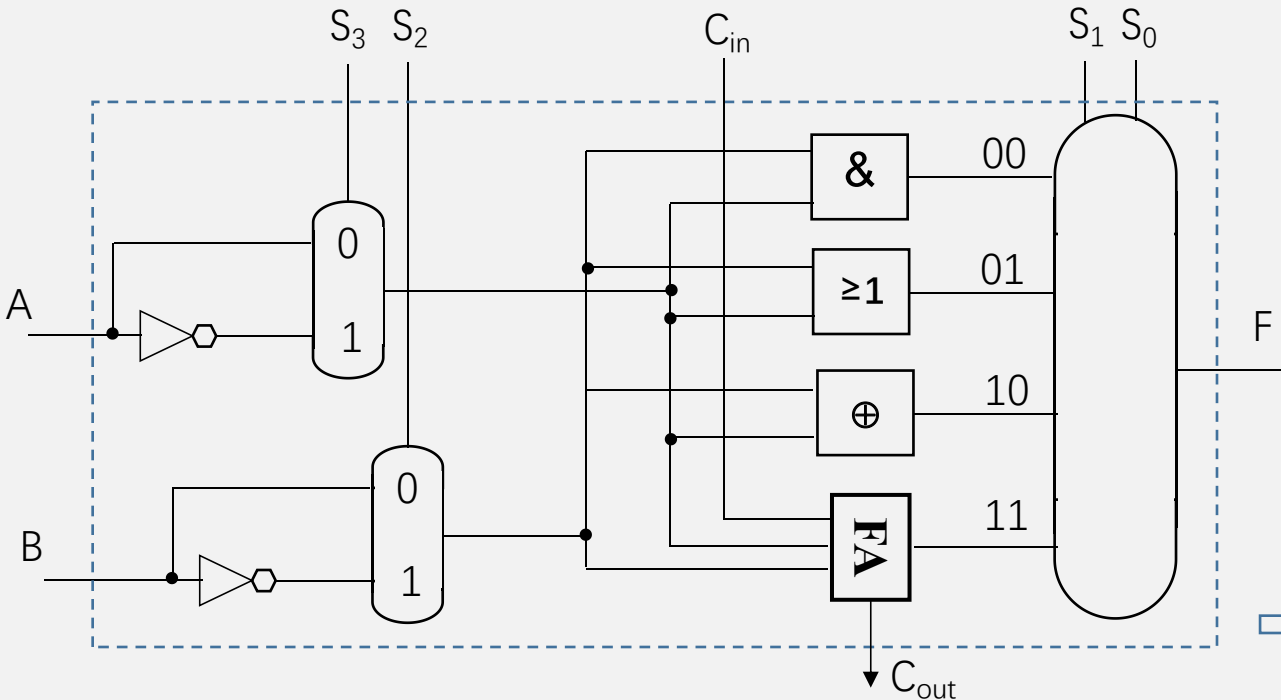
5.1 定点数加/减运算

10. 运算器设计其他方法(结构)

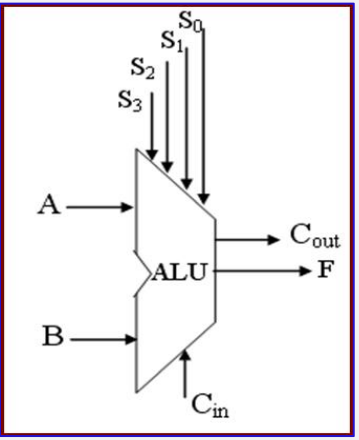


```
if (s== 0,c = a ; else c=b )
```

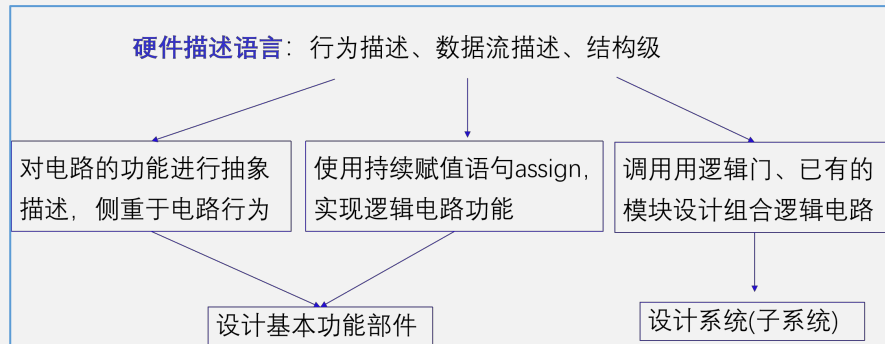
10. 运算器设计其他方法(结构)



- $S_3S_2S_1S_0 = 0000: F = AB$
- $S_3S_2S_1S_0 = 1100: F = \bar{A}\bar{B}$
- $S_3S_2S_1S_0 = 1000: F = \bar{A}B$
- $S_3S_2S_1S_0 = 0100: F = A\bar{B}$
- $S_3S_2S_1S_0 = 0001: F = A+B$
- $S_3S_2S_1S_0 = 0010: F = A \oplus B$
- $S_3S_2S_1S_0 = 0011 \text{ } C_{in}=0: F = A \text{ 加 } B$
- $S_3S_2S_1S_0 = 0111 \text{ } C_{in}=1: F = A \text{ 减 } B$
-



10. 运算器设计其他方法(硬件描述语言: Verilog)

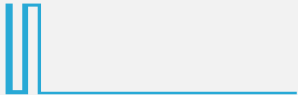


```
moudle add_4 (x,y, sum ,cin, cout )
input[3:0] x,y ;
outpu[3:0] sum ;
output cout ;
input cin ;
sum <= x xor y xor cin;
cout <= (x and y) or ( x and cin ) or ( y and cin )
endmoudle
```

配合FPGA使用(Field Programmable Gate Array)

```
moudle add_4 (x,y, sum ,cin, cout )
input[3:0] x,y ;
output[3:0] sum ;
output cout ;
input cin ;
assign {cout, sum } = x + y + cin;
endmoudle
```

```
module Fadd(x,y,Cin,Cout,Sum);
input x,y,Cin;
output Cout,Sum;
wire a,b,c;
    xor xor1(a,x,y);
    xor2(Sum,a,Cin);
    and and1(b,x,y);
    and2(c,Cin,a);
    or or1(Cout,b,c);
endmodule
```



第五章 第一部分完